

Visual Recognition using Deep Learning

Object Segmentation

林彥宇 教授

Yen-Yu Lin, Professor

國立陽明交通大學 資訊工程學系

Computer Science, National Yang Ming Chiao Tung University

Some slides are modified from Justin Johnson, Andrej Karpathy, Wei-Chih Hung, Cheng-Chun Hsu, and Yun-Chun Chen



Outline

- Semantic segmentation
 - Fully convolutional network (FCN)
 - U-Net
- Instance segmentation
 - Mask R-CNN
- Panoptic segmentation and universal segmentation
 - Mask2Former
- Segmentation with low training data annotation costs
 - Semi-supervised learning
 - Weakly-supervised learning
 - Domain-adaptive training data synthesis



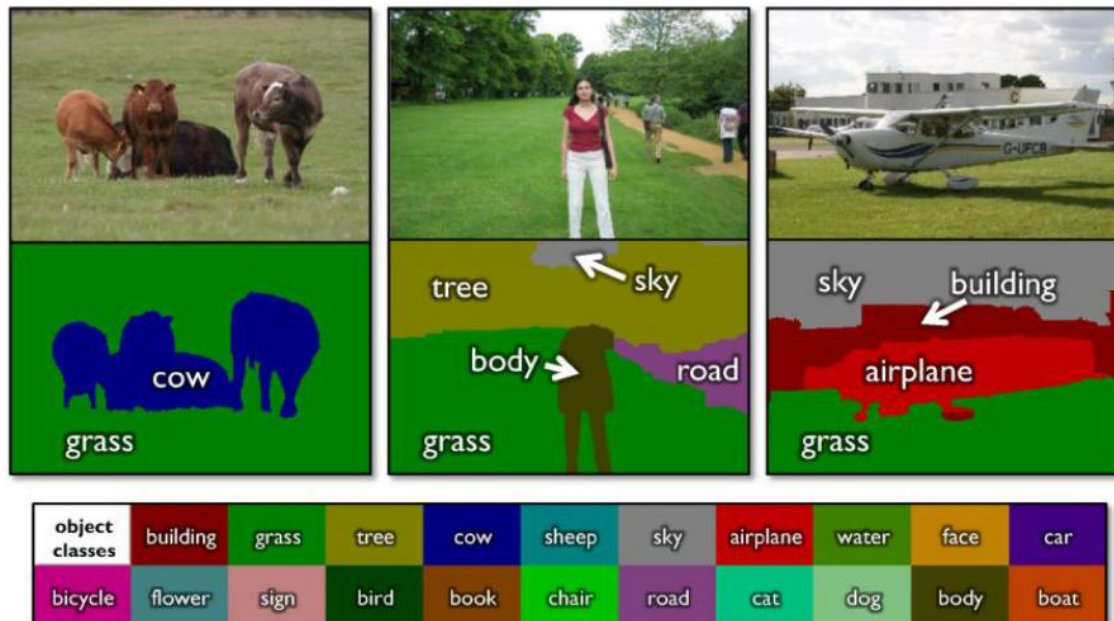
Outline

- Semantic segmentation
 - Fully convolutional network (FCN)
 - U-Net
- Instance segmentation
 - Mask R-CNN
- Panoptic segmentation and universal segmentation
 - Mask2Former
- Segmentation with low training data annotation costs
 - Semi-supervised learning
 - Weakly-supervised learning
 - Domain-adaptive training data synthesis



Semantic segmentation

- Goal: Label each pixel to one of predefined classes (or background)
- Critical to high-level vision tasks such as scene understanding, robot navigation, and image retrieval

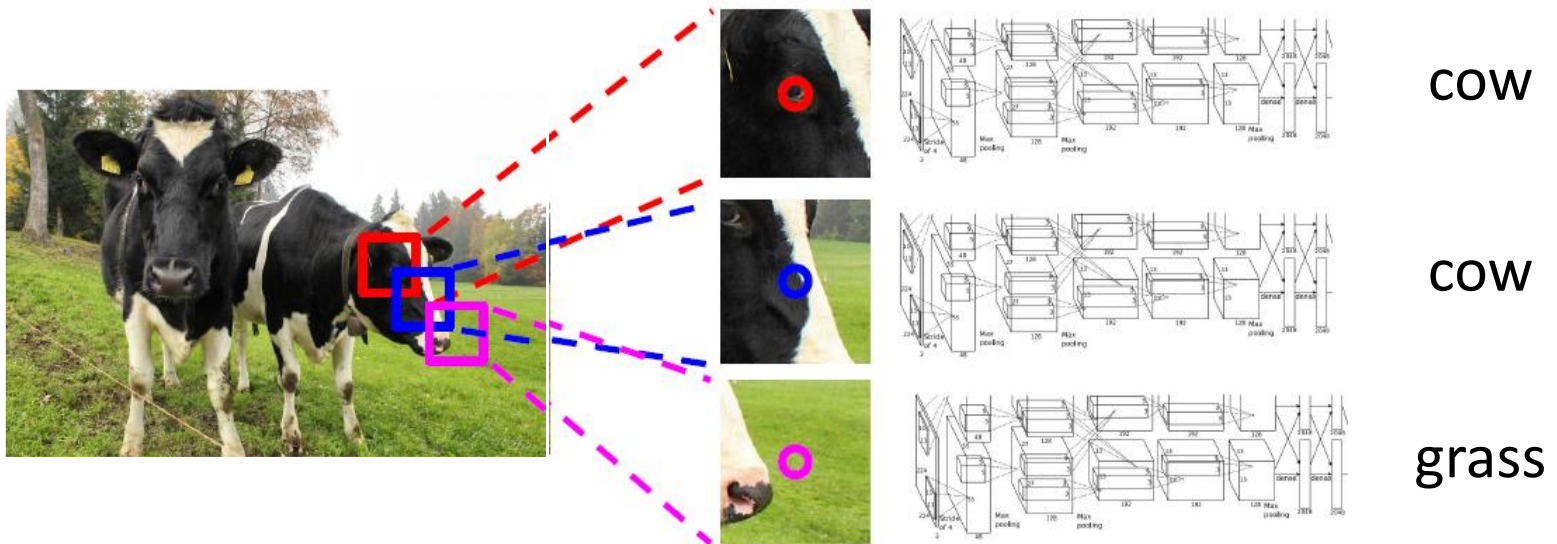


[Shotton et al., 2007]



Sliding window for pixel-wise classification

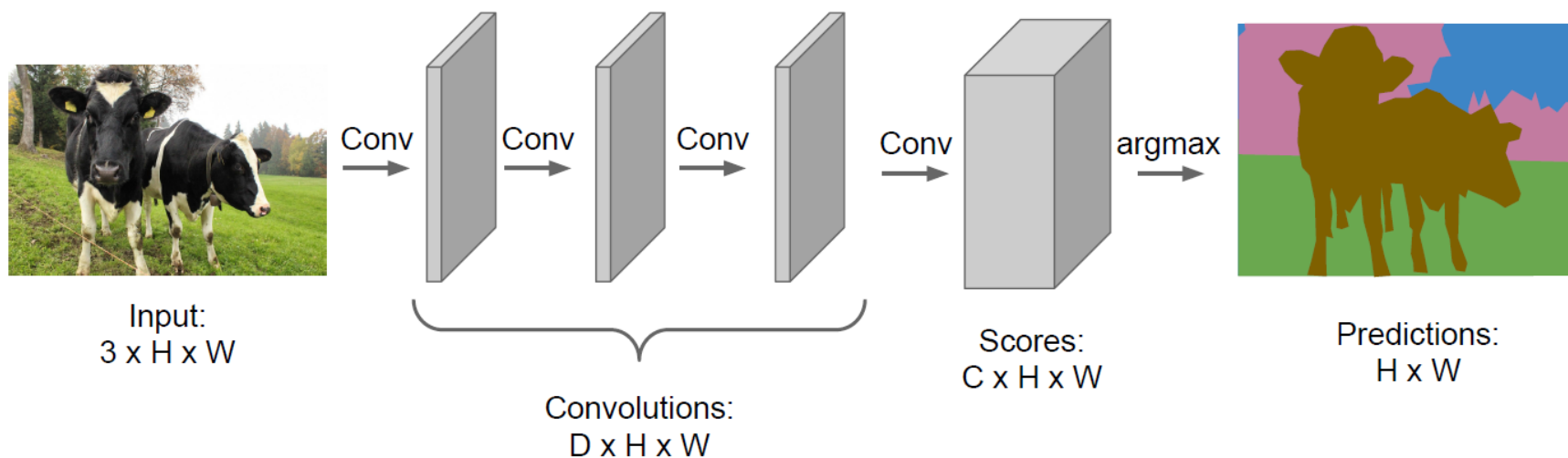
- CNN is powerful for classification
- For classifying each pixel, apply CNN to the patch centered at that pixel



- Too inefficient

Idea for semantic segmentation: **Full convolution**

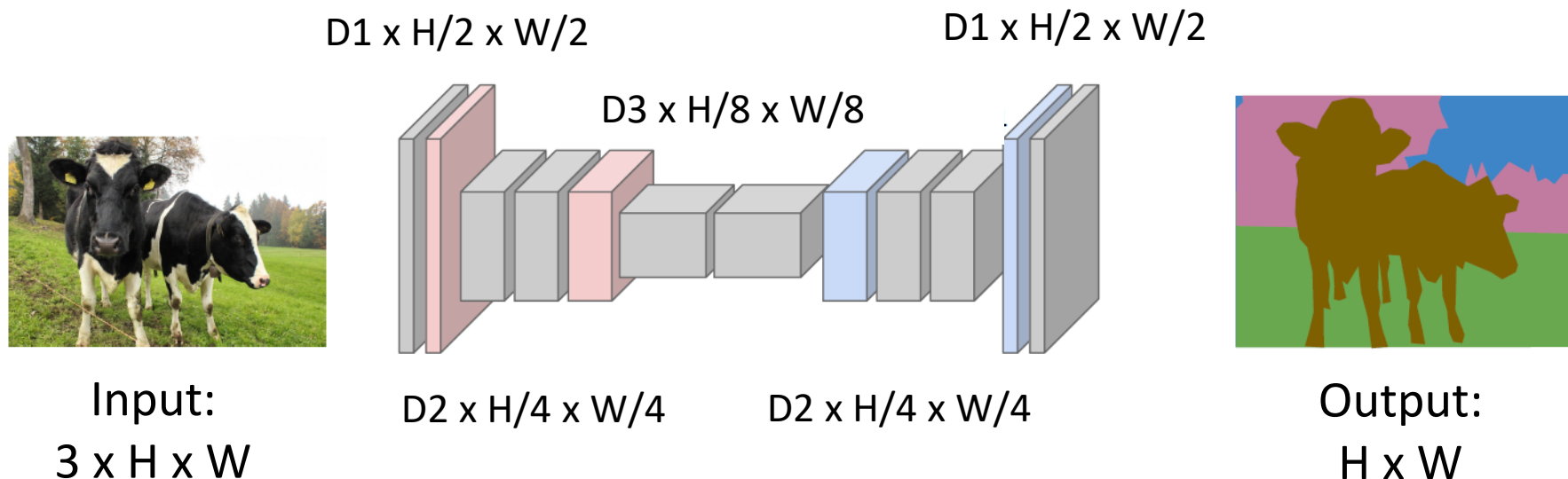
- An observation: pixels in, pixels out
- Convolutions with padding preserve the resolution
- Apply softmax operation for each pixel in the last layer



- **Not very efficient: Convolution at a high-resolution**

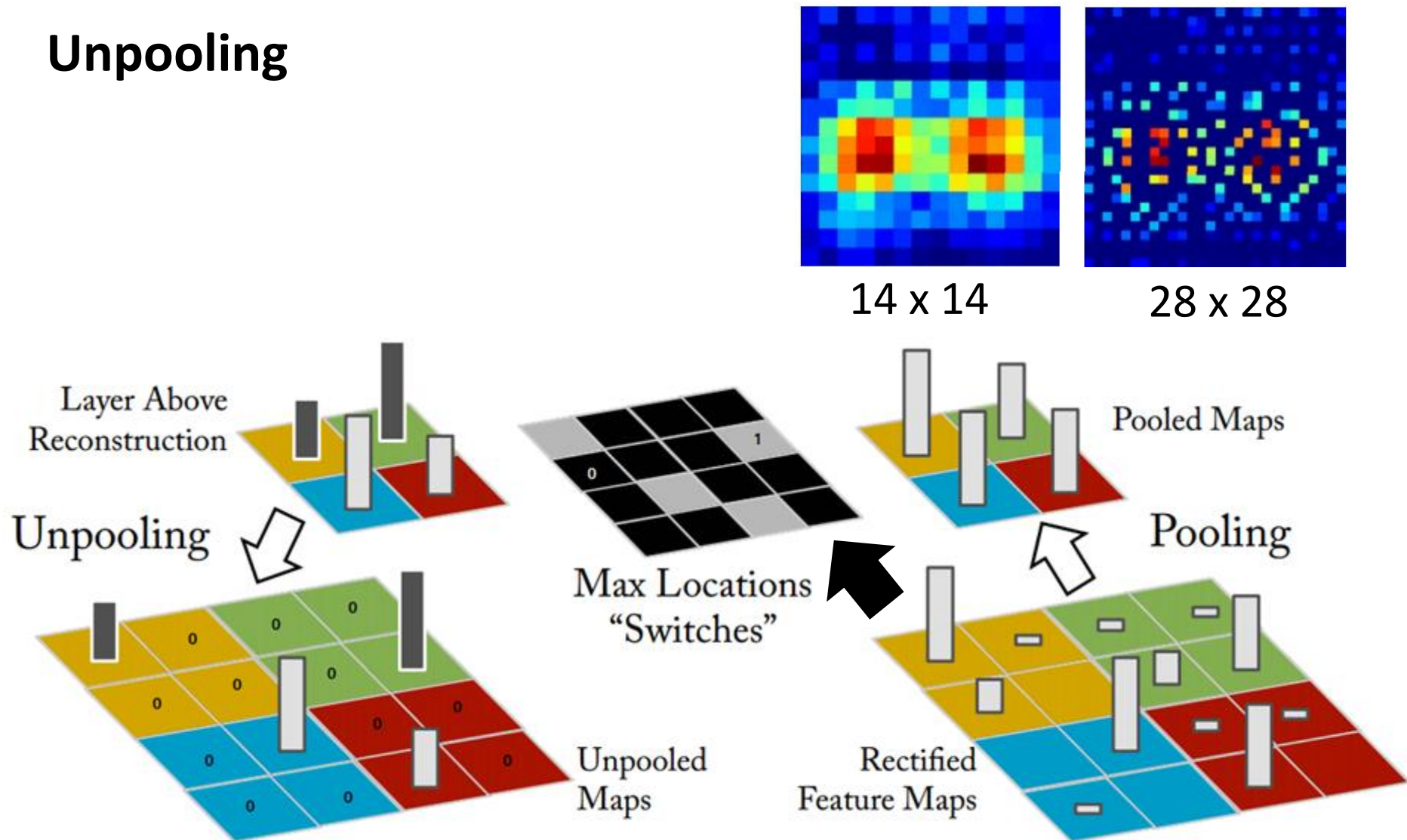
Idea for semantic segmentation: Full convolution

- A network of convolutional layers, with **downsampling** and **upsampling** inside the network



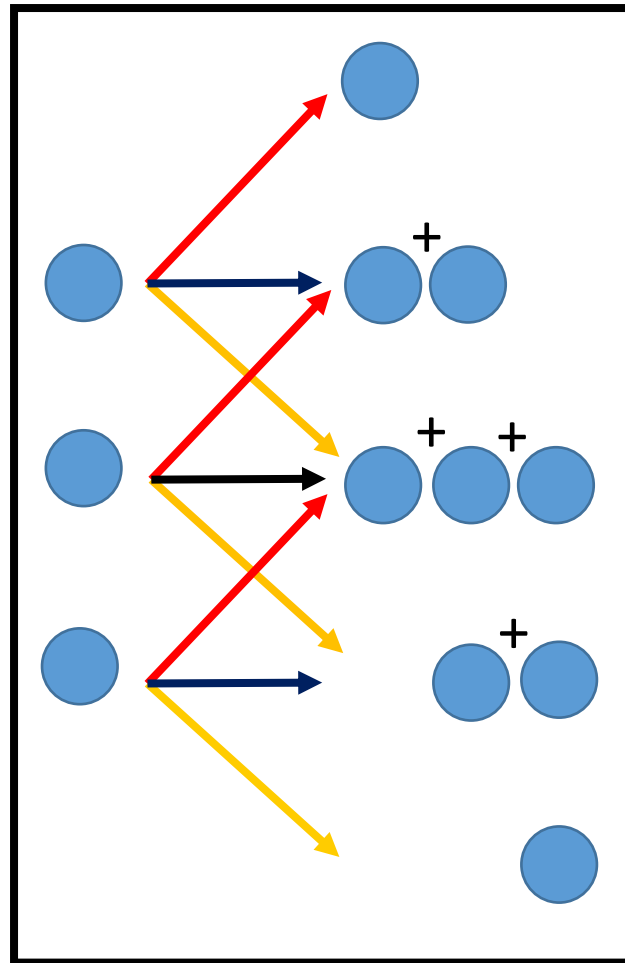
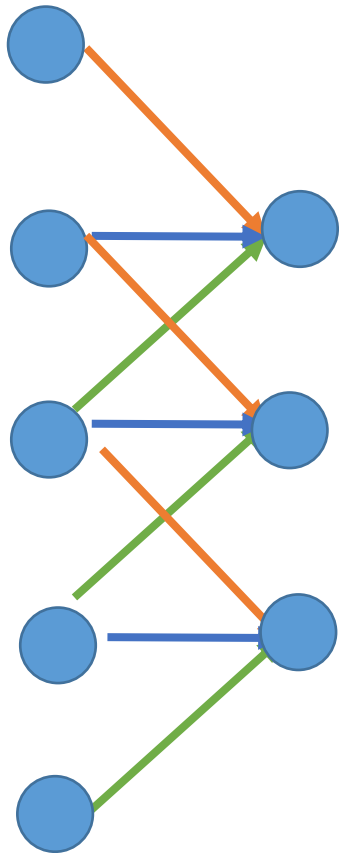
- Downsampling**: Pooling or strided convolution
- Upsampling**: How to increase feature map resolutions?

Unpooling

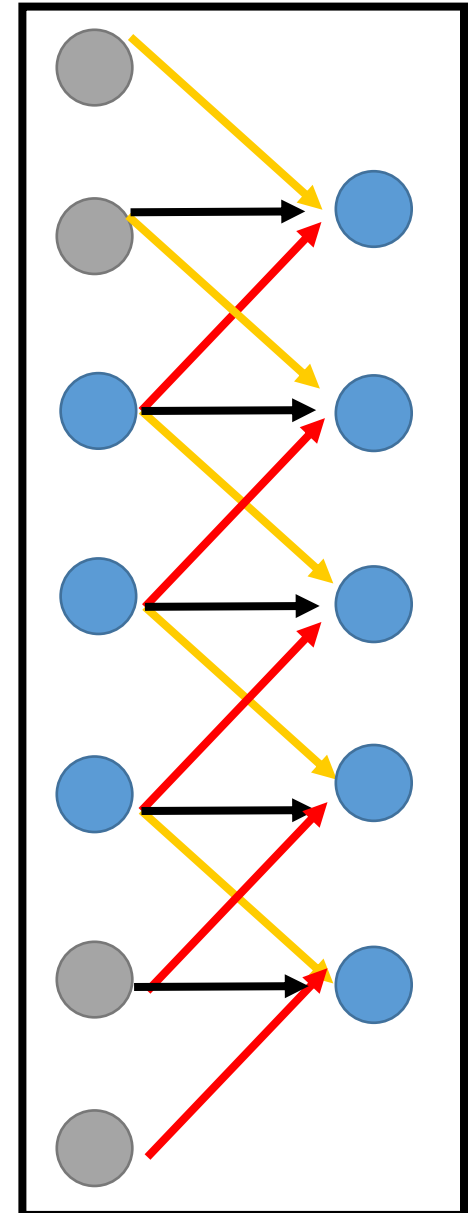


Deconvolution

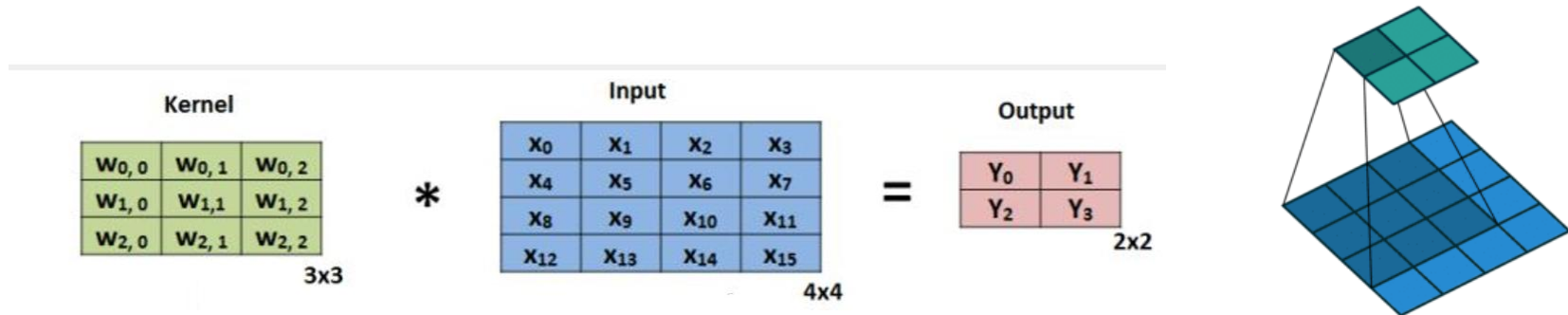
Deconvolution is (transposed) convolution.



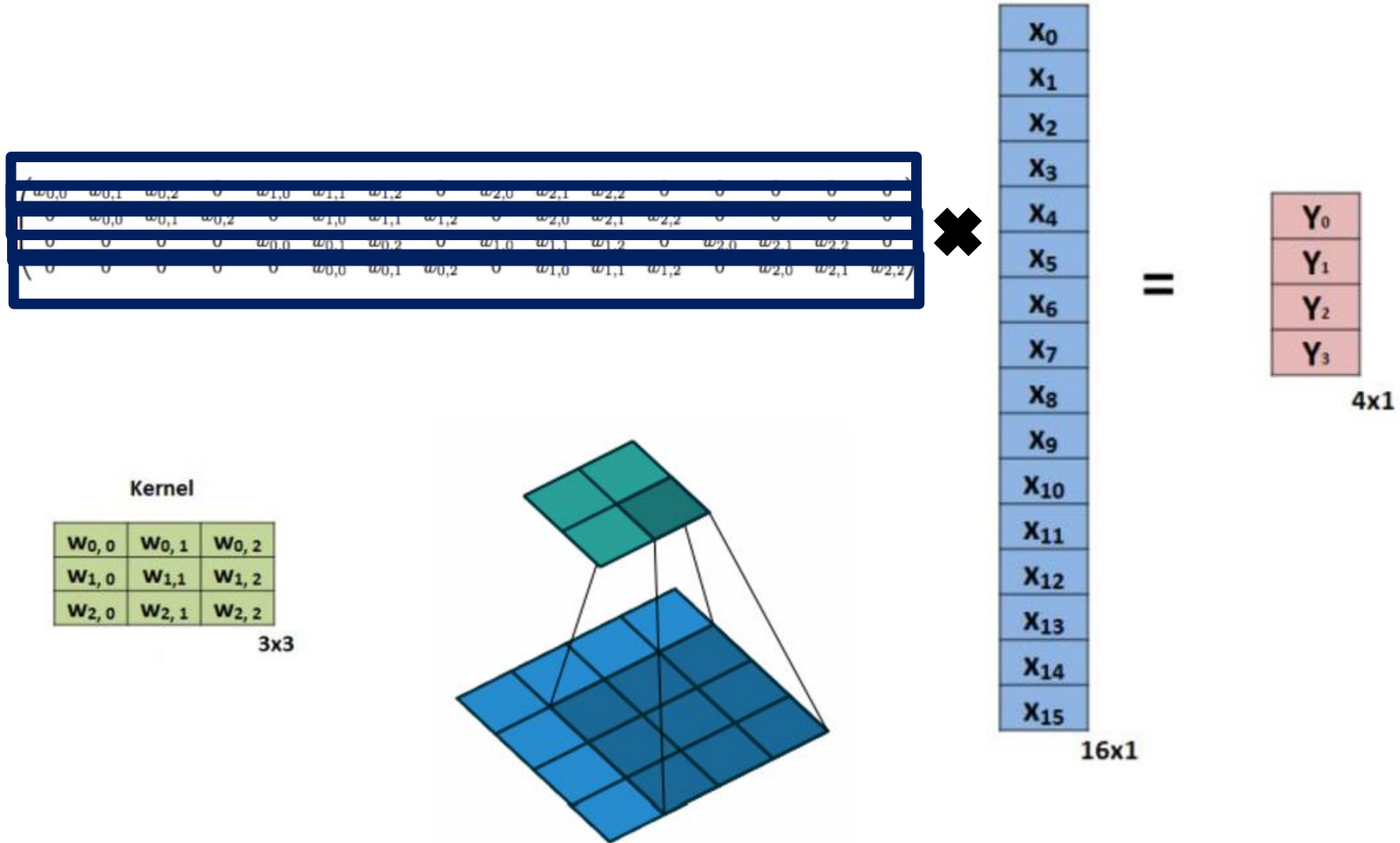
=



Convolution

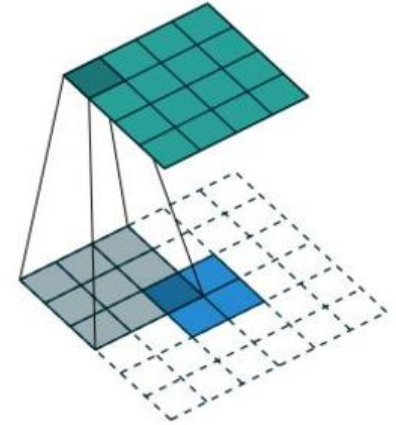


Convolution in form of matrix multiplication



Transposed convolution

$$\begin{pmatrix}
 w_{0,0} & 0 & 0 & 0 \\
 w_{0,1} & w_{0,0} & 0 & 0 \\
 w_{0,2} & w_{0,1} & 0 & 0 \\
 0 & w_{0,2} & 0 & 0 \\
 w_{1,0} & 0 & w_{0,0} & 0 \\
 w_{1,1} & w_{1,0} & w_{0,1} & w_{0,0} \\
 w_{1,2} & w_{1,1} & w_{0,2} & w_{0,1} \\
 0 & w_{1,2} & 0 & w_{0,2} \\
 w_{2,0} & 0 & w_{1,0} & 0 \\
 w_{2,1} & w_{2,0} & w_{1,1} & w_{1,0} \\
 w_{2,2} & w_{2,1} & w_{1,2} & w_{1,1} \\
 0 & w_{2,2} & 0 & w_{1,2} \\
 0 & 0 & w_{2,0} & 0 \\
 0 & 0 & w_{2,1} & w_{2,0} \\
 0 & 0 & w_{2,2} & w_{2,1} \\
 0 & 0 & 0 & w_{2,2}
 \end{pmatrix}
 \times
 \begin{pmatrix}
 Y_0 \\
 Y_1 \\
 Y_2 \\
 Y_3
 \end{pmatrix}_{4 \times 1}
 =
 \begin{pmatrix}
 x_0 \\
 x_1 \\
 x_2 \\
 x_3 \\
 x_4 \\
 x_5 \\
 x_6 \\
 x_7 \\
 x_8 \\
 x_9 \\
 x_{10} \\
 x_{11} \\
 x_{12} \\
 x_{13} \\
 x_{14} \\
 x_{15}
 \end{pmatrix}_{16 \times 1}$$

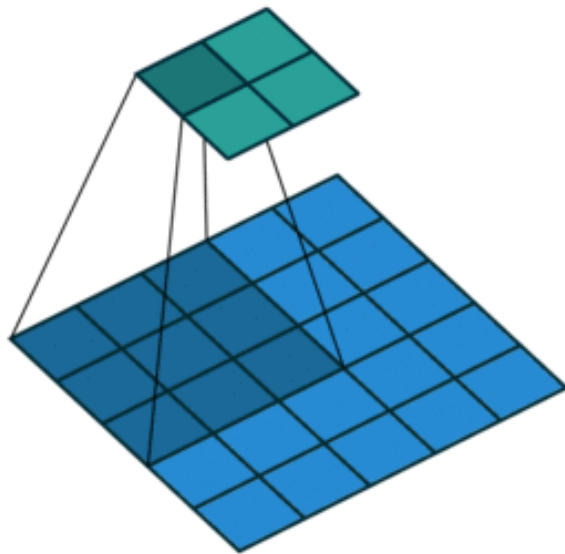


Kernel

W2,2	W2,1	W2,0
W1,2	W1,1	W1,0
W0,2	W0,1	W0,0



Convolution with a non-unit stride



stride = 2

o = output size (2×2)

s = strides (2)

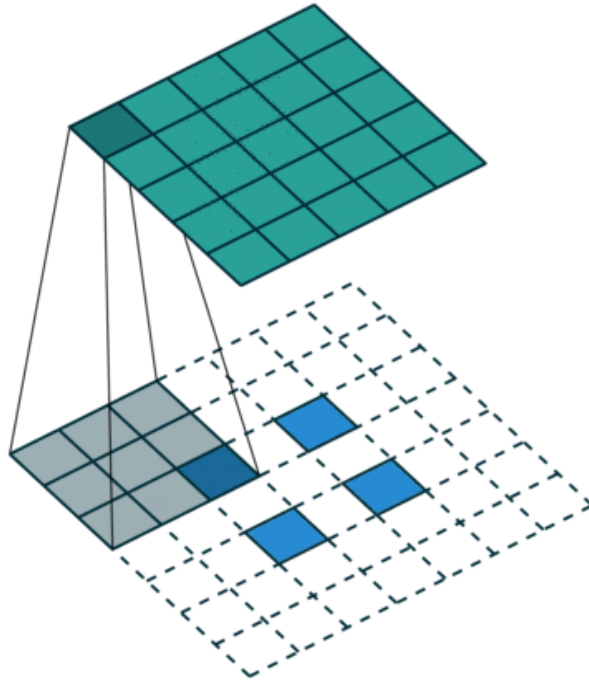
i = input size (5×5)

k = kernel size (3×3)

$$o = \left\lfloor \frac{i - k}{s} \right\rfloor + 1.$$



Transposed convolution with a non-unit stride



stride = 2

o' = output size

s = strides

i' = input size

k = kernel size

$$o' = s(i' - 1) + k.$$



Remarks: Transposed convolution

- A transformation like the decoding layer of a convolutional encoder
- It can learn the projection from low-resolution feature maps to high-resolution ones
- **Skip connections** are often adopted to work with transposed convolutions



Outline

- Semantic segmentation
 - Fully convolutional network (FCN)
 - U-Net
- Instance segmentation
 - Mask R-CNN
- Panoptic segmentation and universal segmentation
 - Mask2Former
- Segmentation with low training data annotation costs
 - Semi-supervised learning
 - Weakly-supervised learning
 - Domain-adaptive training data synthesis



Fully Convolutional Networks (FCN) for semantic segmentation

[Long et al. CVPR'15]

- Fully convolutional networks
 - There are only convolution layer and transposed convolution layer in the network architecture
- Modify from convolution nets for classification
 - The networks are pre-trained on other datasets such as ImageNet for classification and then fine tuned with datasets for semantic segmentation such as Pascal VOC



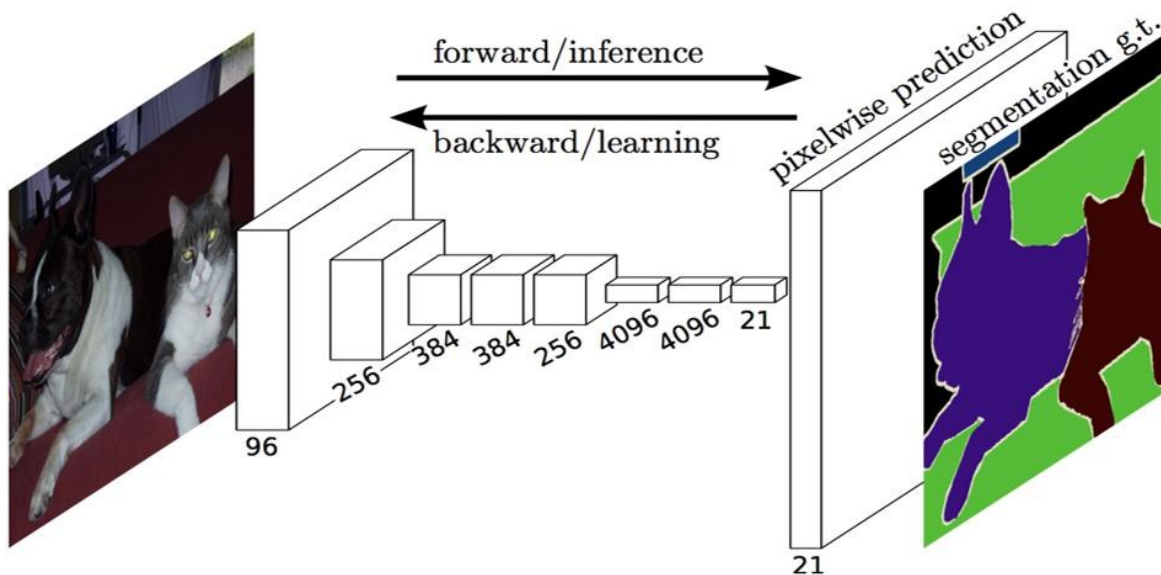
input image from Pascal VOC



segmentation results

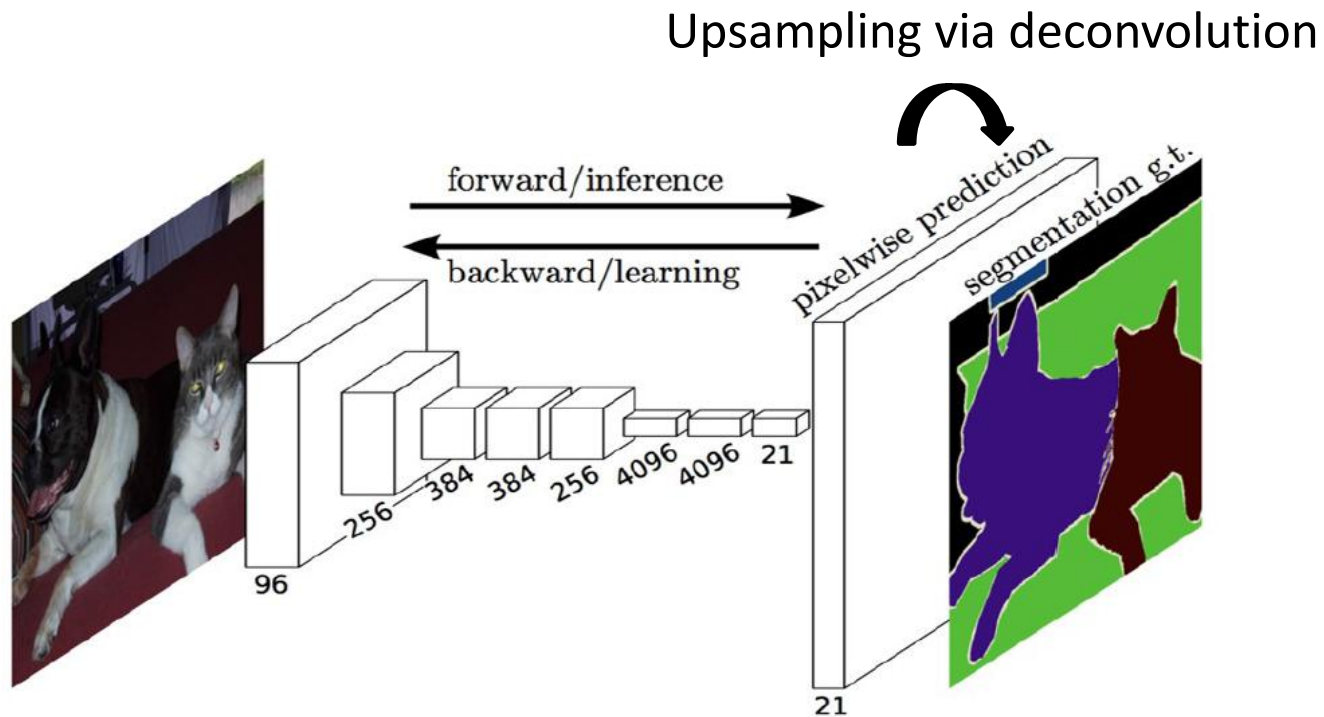
Approach

- Start from AlexNet, VGG 16-layer net, or GoogLeNet
- Discard the final classifier layer
- Append a 1×1 convolution with channel dimension 21 to predict scores for each of the PASCAL classes

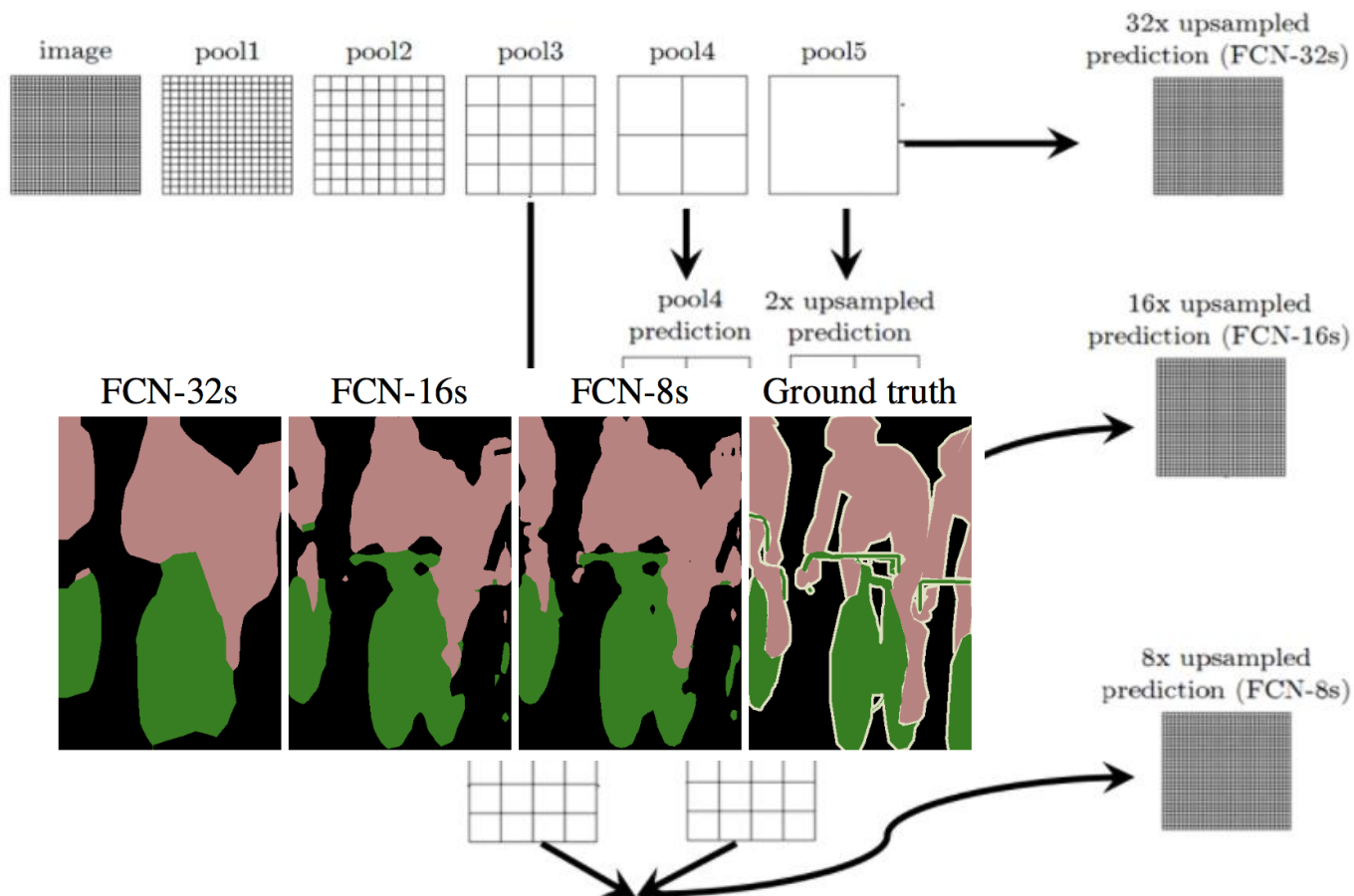


Fully convolutional network (FCN)

- FCN learns to make dense predictions for per-pixel tasks, such as semantic segmentation

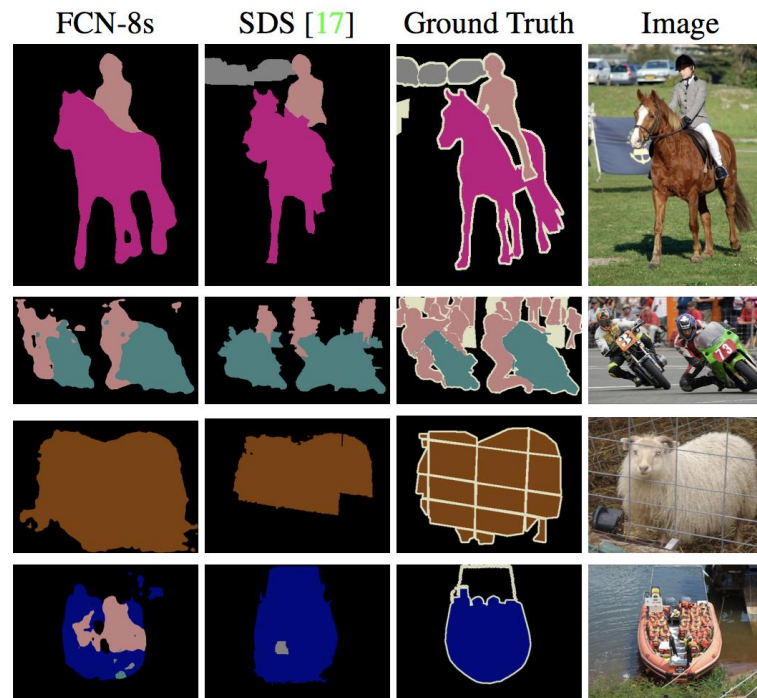


Fully convolutional network (FCN)



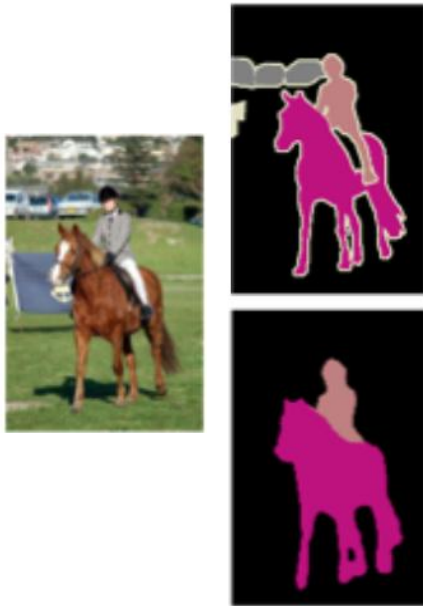
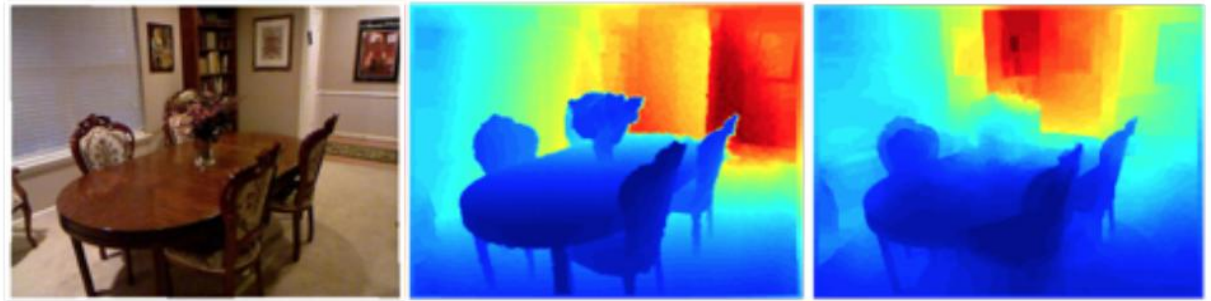
Experimental Results on Pascal VOC

	mean IU VOC2011 test	mean IU VOC2012 test	inference time
R-CNN [12]	47.9	-	-
SDS [17]	52.6	51.6	~ 50 s
FCN-8s	62.7	62.2	~ 175 ms



More applications supported by FCN

depth prediction



semantic
segmentation



boundary prediction

Outline

- Semantic segmentation
 - Fully convolutional network (FCN)
 - U-Net
- Instance segmentation
 - Mask R-CNN
- Panoptic segmentation and universal segmentation
 - Mask2Former
- Segmentation with low training data annotation costs
 - Semi-supervised learning
 - Weakly-supervised learning
 - Domain-adaptive training data synthesis

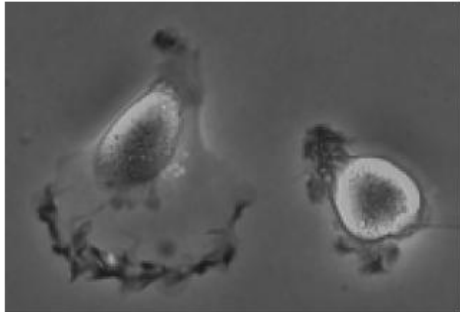


Introduction

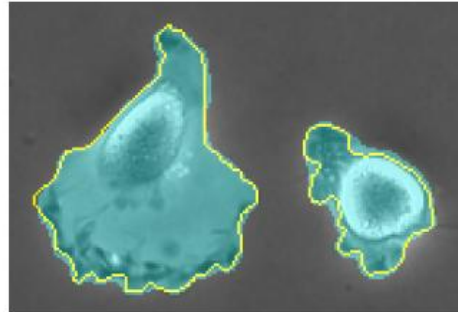
[Ronneberger et al. MICCAI'15]

- In biomedical image processing
 - Localization is critical
 - Available training images are insufficient

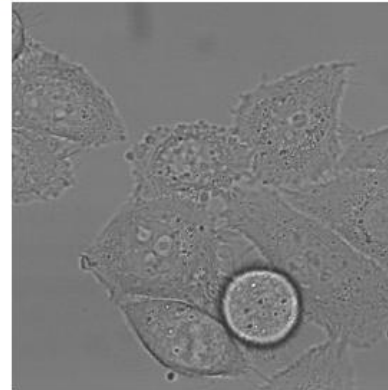
a



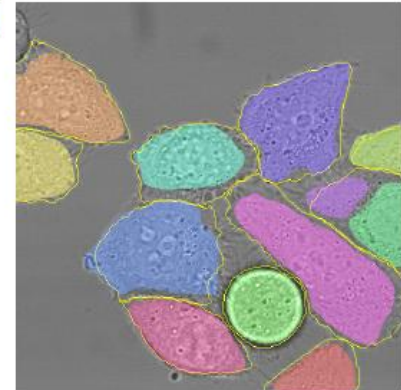
b



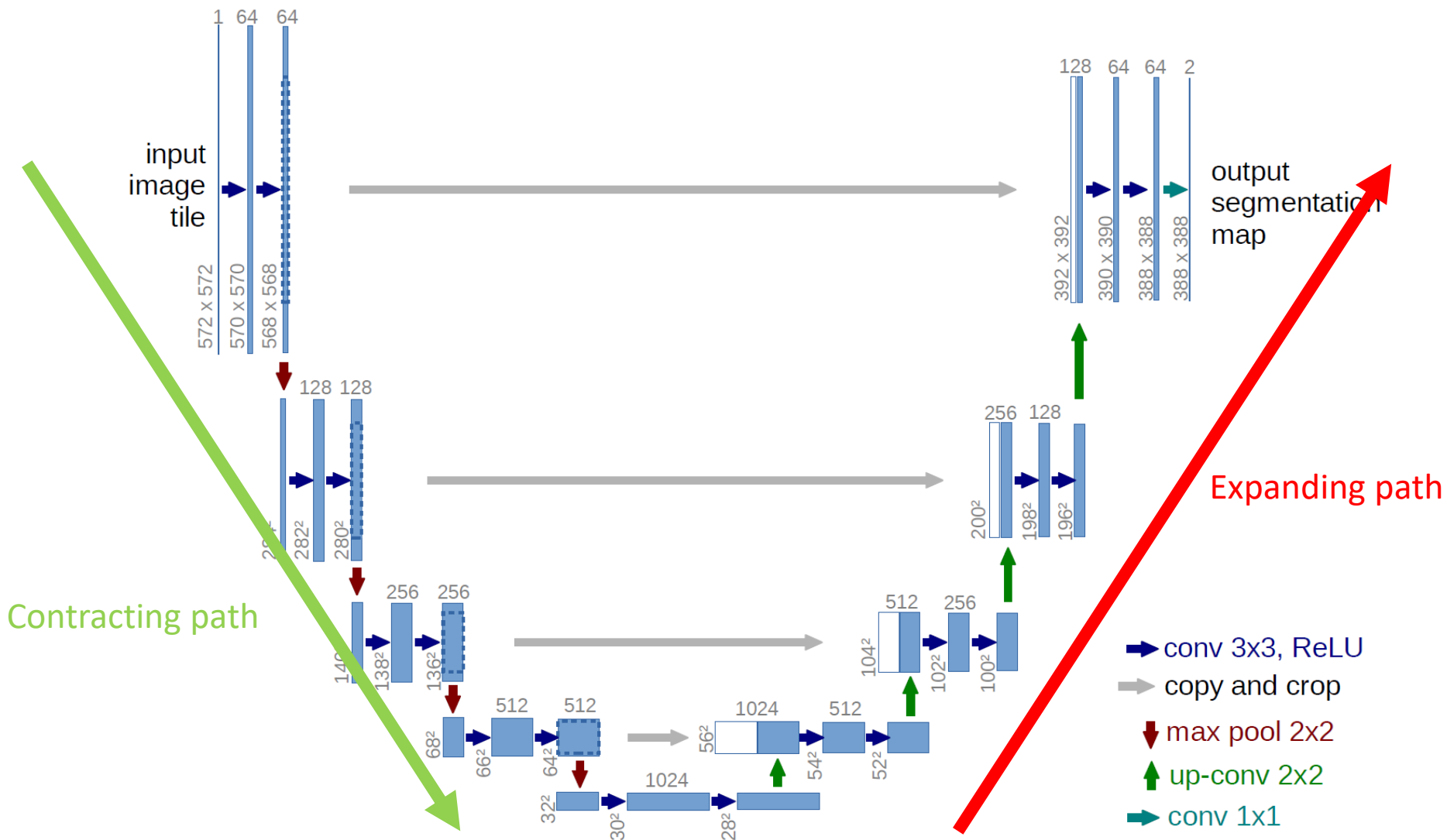
c



d



Network Architecture



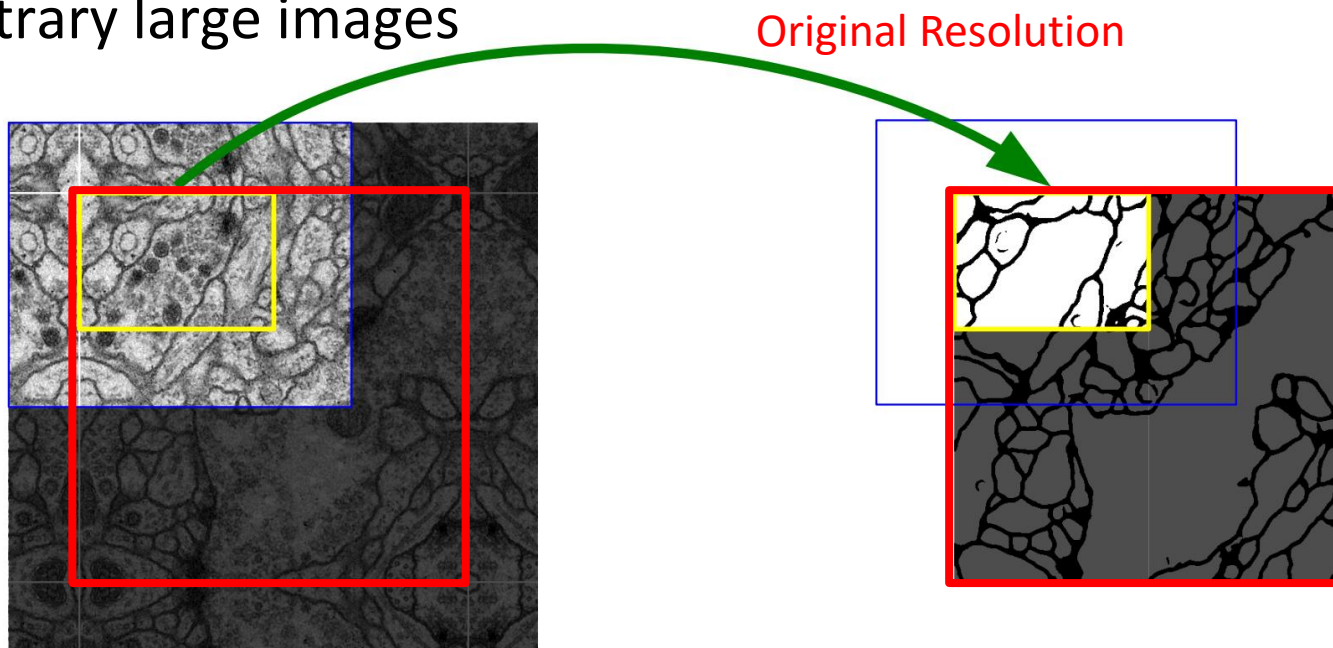
Network architecture

- Consist of a **contracting path** and an **expanding path**
 - **Contracting path**: Conv. + max pooling
 - **Expanding path**: “Up-conv” + concatenation + Conv.
 - The final layer has a 1x1 conv. to predict the class



Pre-processing

- Enlarge the input image by mirroring outside the border in order to fit the desired output resolution
- Acquire better context information
- Overlap-Tile Strategy: allow seamless segmentation of arbitrary large images



Energy function

- Weighted cross entropy loss with pixel-wise soft-max

$$p_k(x) = \frac{e^{a_k(x)}}{\sum_{k'=1}^K e^{a_{k'}(x)}}$$

K : number of classes

k : index of channels in the last feature map

x : pixel position

$$E = \sum_{\mathbf{x} \in \Omega} w(\mathbf{x}) \log(p_{\ell(\mathbf{x})}(\mathbf{x}))$$

$\ell(\cdot)$: true label of a pixel

$w(\cdot)$: weight from pre-defined weight map

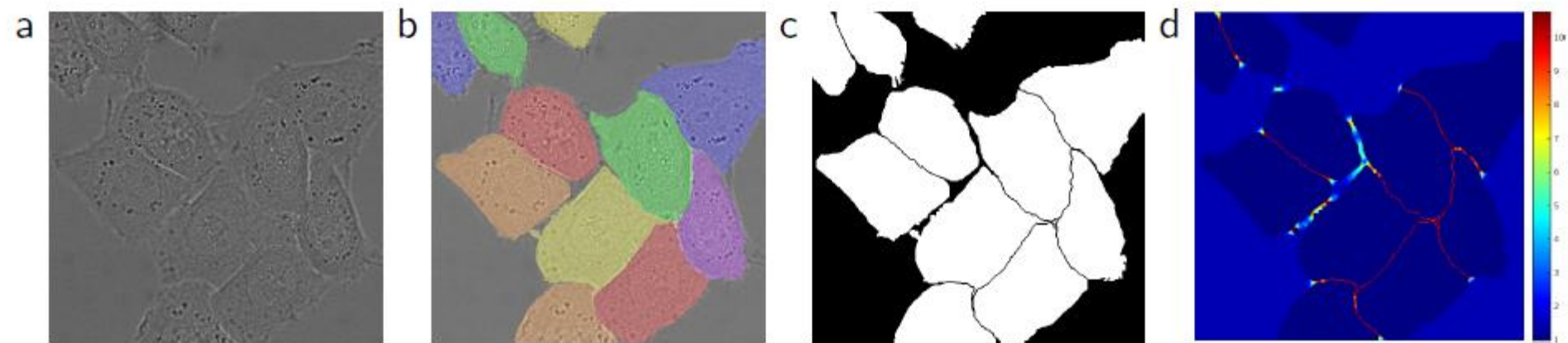


Weight map

- A difficulty of the task is to identify individual cells
- The map is pre-defined to balance class frequency and force to learn the separation area between cells

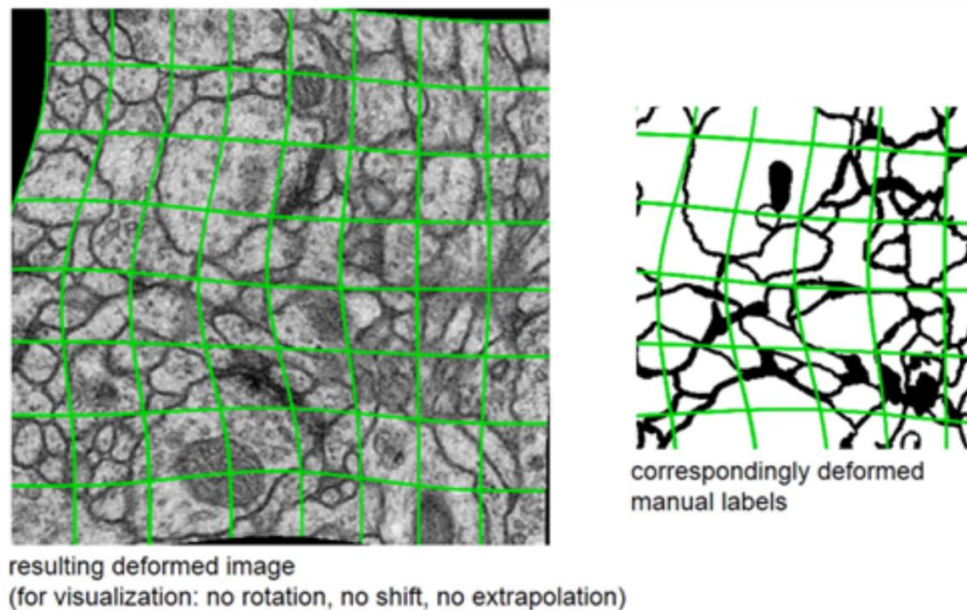
$$w(\mathbf{x}) = w_c(\mathbf{x}) + w_0 \cdot \exp \left(-\frac{(d_1(\mathbf{x}) + d_2(\mathbf{x}))^2}{2\sigma^2} \right)$$

$w_c(\cdot)$: class weight; $d_1(\cdot)$: the distance to the border of the nearest cell;
 $d_2(\cdot)$: the distance to the border of the second nearest cell; w_0, σ : hyperparameters



Elastic deformation

- Data augmentation is critical for U-Net due to few available annotated images
- Elastic deformation is to randomly deform the input image and its corresponding segmentation map



[Image Source] [Review: U-Net \(Biomedical Image Segmentation\)](#)

[Code] [elastic_distortion.py](#)



ISBI cell tracking challenge 2014 and 2015

Name	PhC-U373	DIC-HeLa
IMCB-SG (2014)	0.2669	0.2935
KTH-SE (2014)	0.7953	0.4607
HOUS-US (2014)	0.5323	-
second-best 2015	0.83	0.46
u-net (2015)	0.9203	0.7756

Metric: IOU



Outline

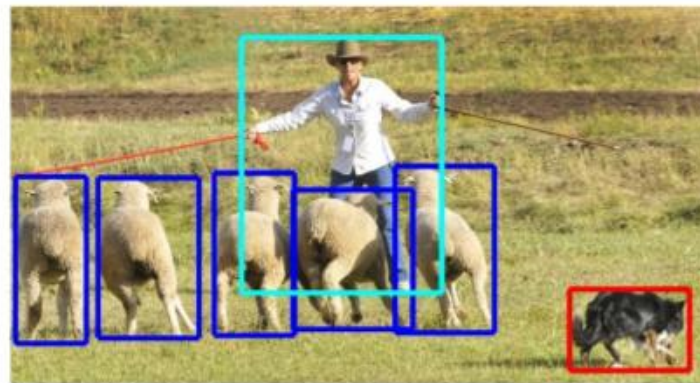
- Semantic segmentation
 - Fully convolutional network (FCN)
 - U-Net
- Instance segmentation
 - Mask R-CNN
- Panoptic segmentation and universal segmentation
 - Mask2Former
- Segmentation with low training data annotation costs
 - Semi-supervised learning
 - Weakly-supervised learning
 - Domain-adaptive training data synthesis



Instance segmentation



(a) Image classification



(b) Object localization



(c) Semantic segmentation



(d) Instance segmentation

Outline

- Semantic segmentation
 - Fully convolutional network (FCN)
 - U-Net
- Instance segmentation
 - Mask R-CNN
- Panoptic segmentation and universal segmentation
 - Mask2Former
- Segmentation with low training data annotation costs
 - Semi-supervised learning
 - Weakly-supervised learning
 - Domain-adaptive training data synthesis



Mask R-CNN

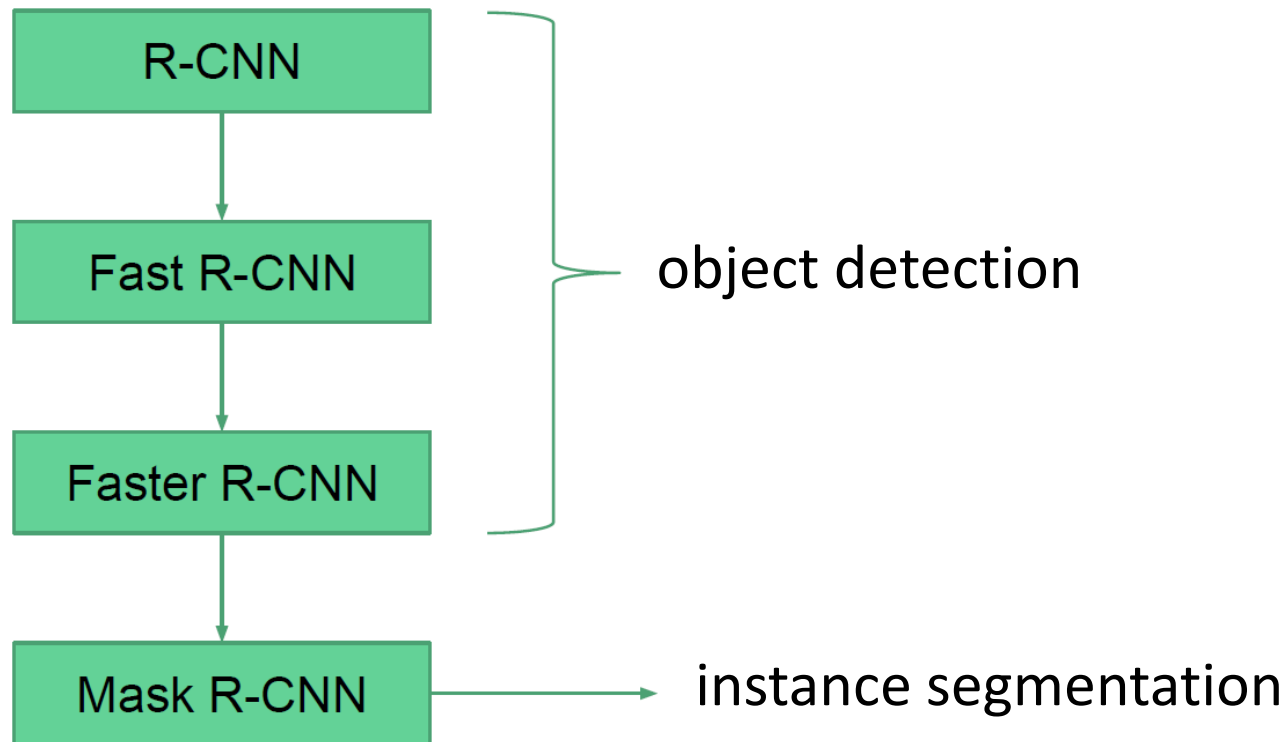
[He et al. ICCV'17]

- Instance segmentation
 - Correct detection of all objects
 - Precisely segment each detected object
 - Object detection + Semantic segmentation
- Mask R-CNN
 - Faster R-CNN + FCN
 - Faster R-CNN for object detection
 - FCN for semantic segmentation



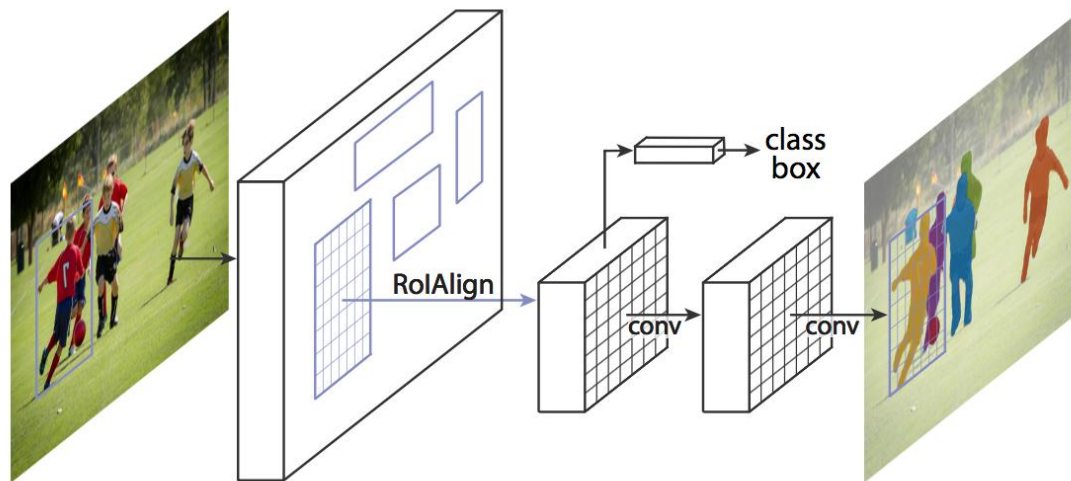
Upper : input image
Lower : output image
Image: Pascal VOC 2012

R-CNN family

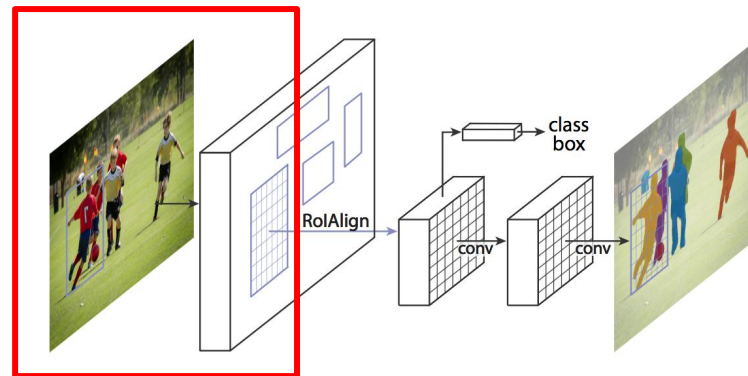


Overview of Mask R-CNN

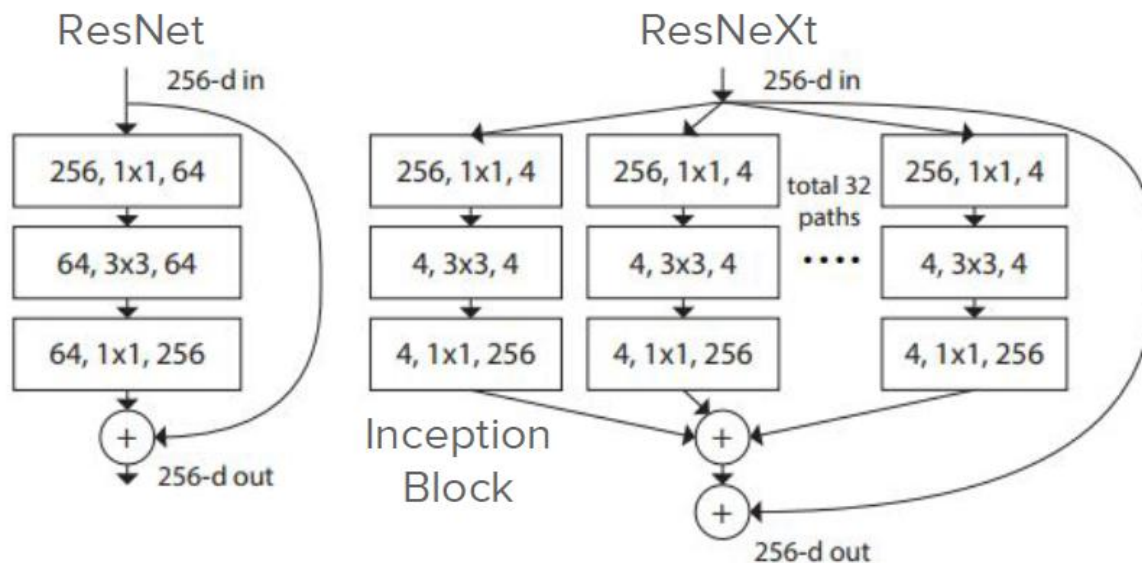
- Use ResNet or feature pyramid network to get feature maps
- Use region proposal network (RPN) to get proposals
- Use RoIAlign rather than RoI pooling to resize each proposal
- Use one detection branch for box classification and regression
- Use one segmentation branch for box segmentation



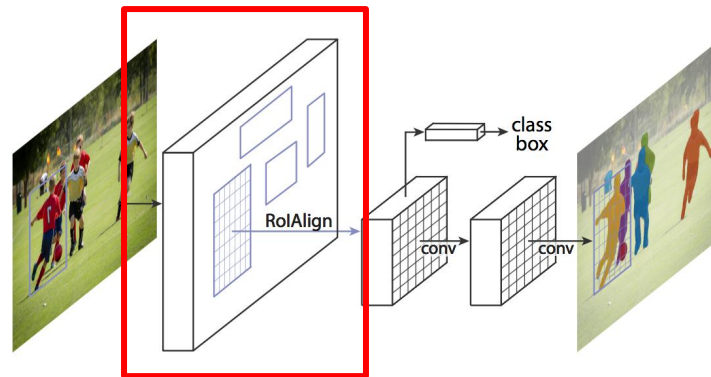
Feature extractor



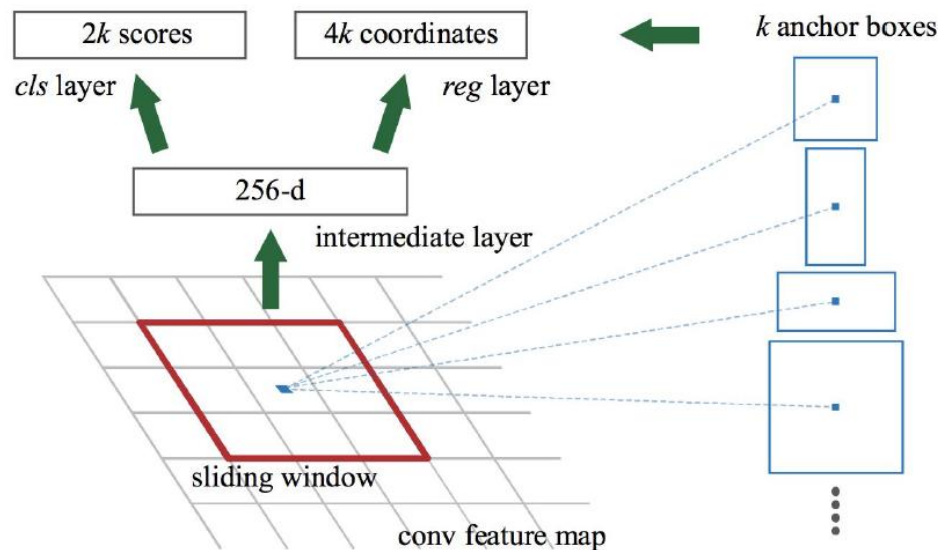
- The authors of Mask R-CNN try two network settings
 - ResNet
 - Feature pyramid network (FPN) with ResNet or ResNeXt



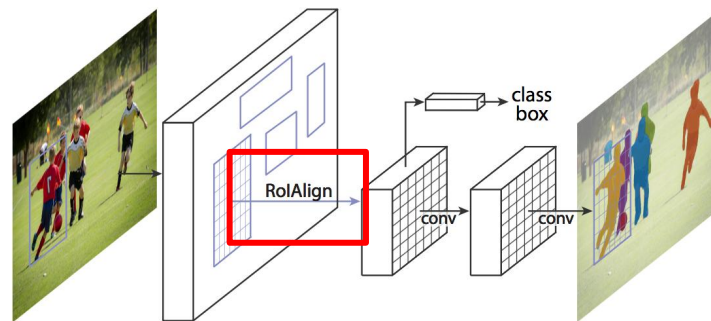
Region proposal network



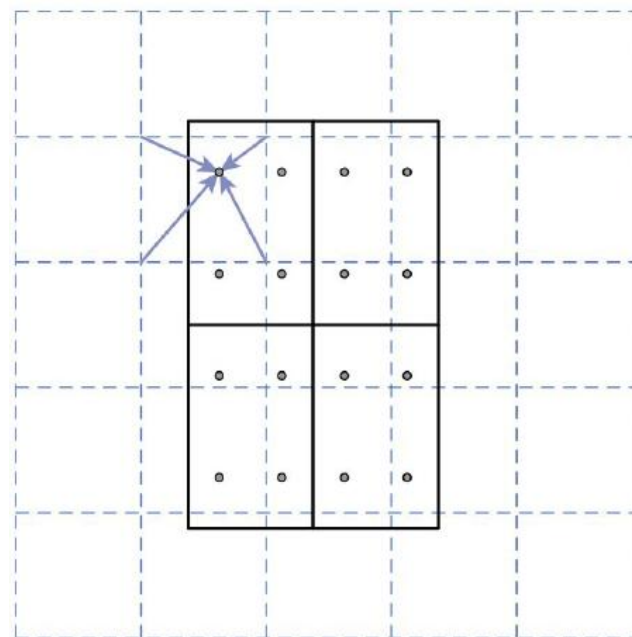
- Input: computed feature maps by the feature extractor
- Output: a set of proposals



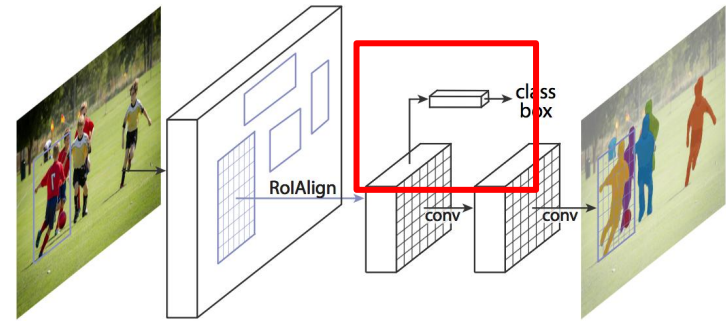
RolAlign



- Why RolAlign or Rol pooling
 - Resize all the proposals to **the same resolution** so that the same sub-networks can be used to predict **class, mask and regress bounding box**
- RolAlign
 - Use **bilinear interpolation** to sample 4 points in one bin
 - Perform **max pooling** to select a point to represent that bin

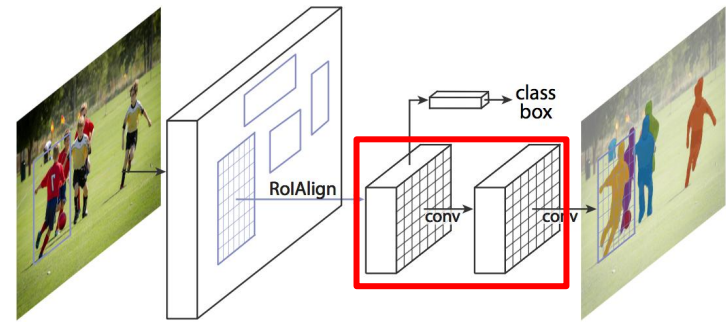


Detection branch



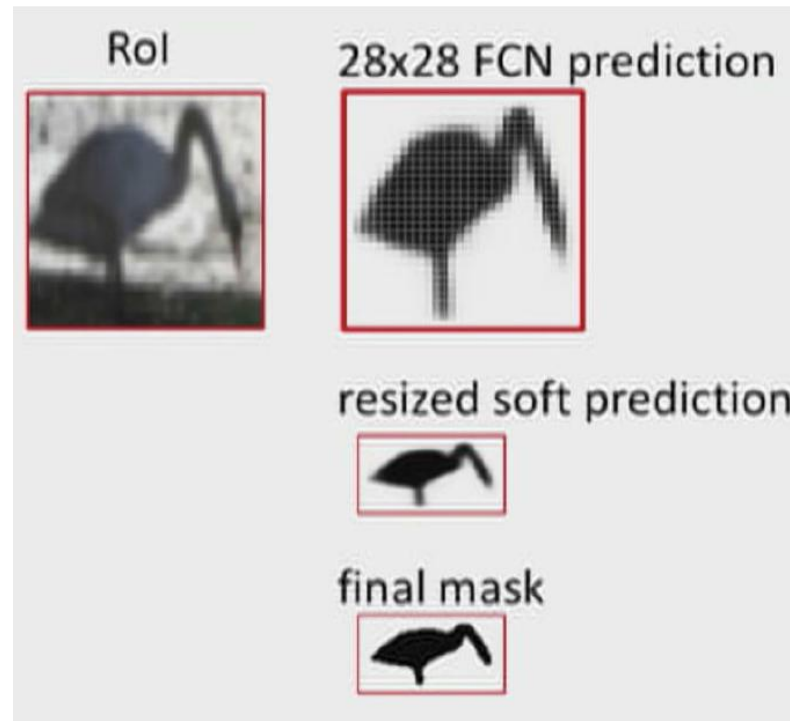
- Class prediction
 - Predict the class of each proposal
- Box regression
 - Further refine the location of each proposal to get a more accurate box

Segmentation branch



- Binary mask prediction

- Apply FCN to each proposal
- Object mask is resized to fit the resolution of original proposal



Network architecture of Mask-RCNN

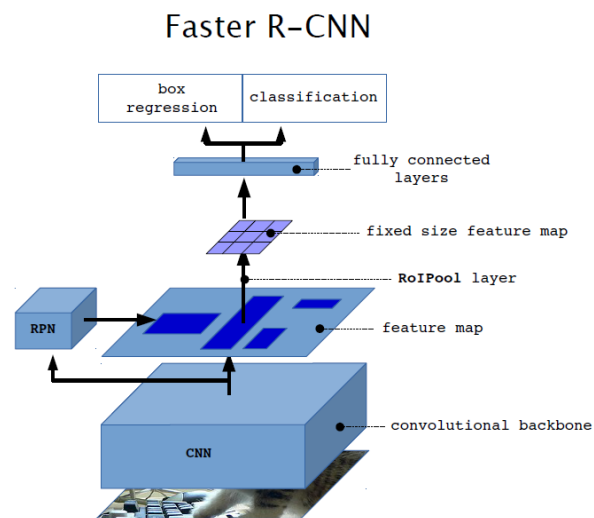
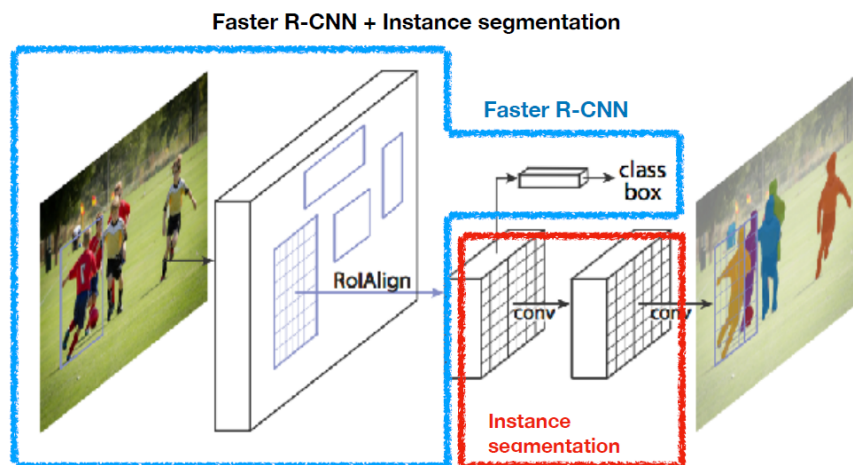
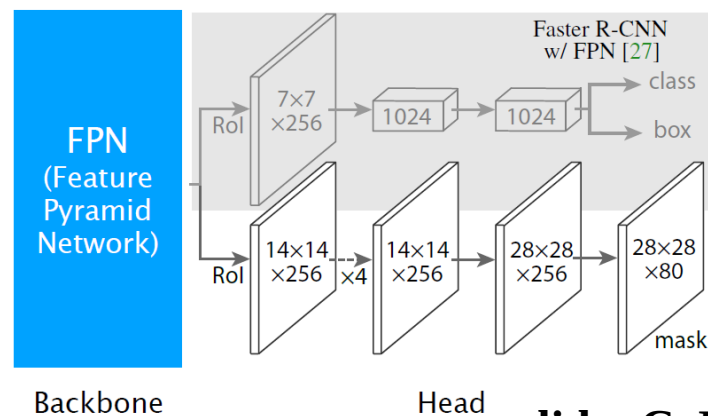
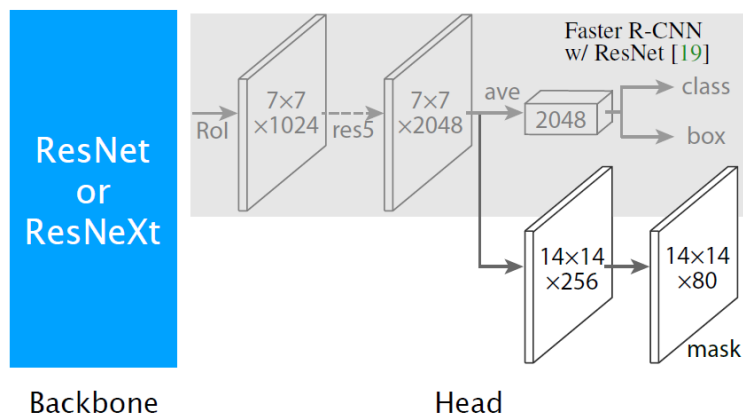
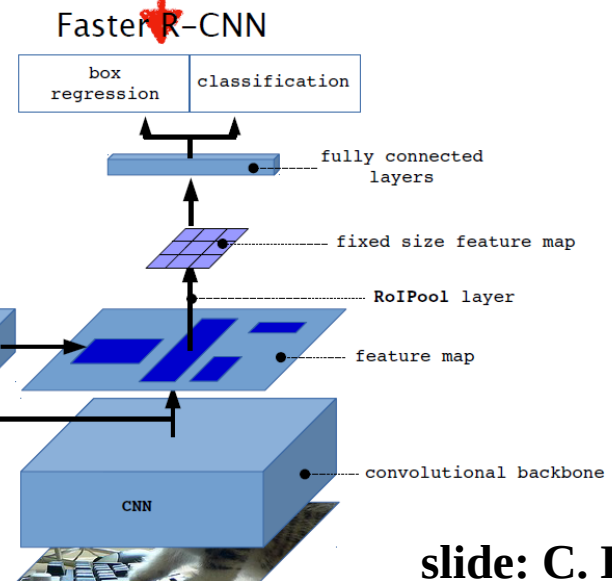
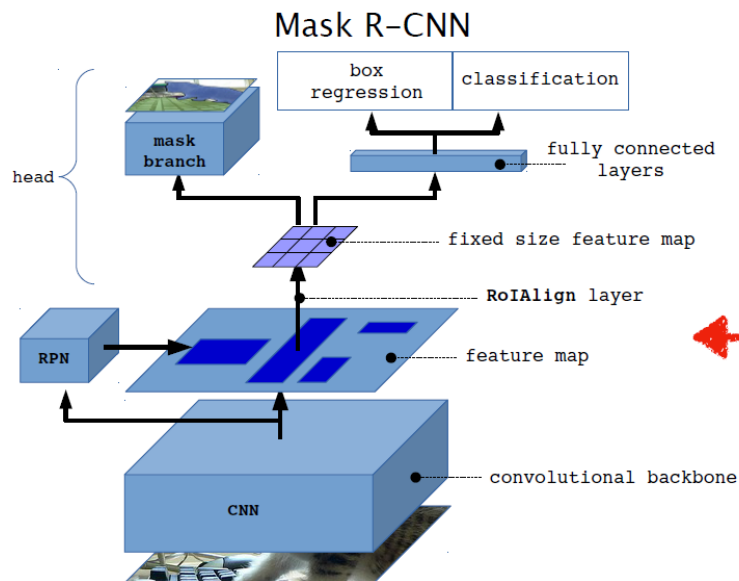
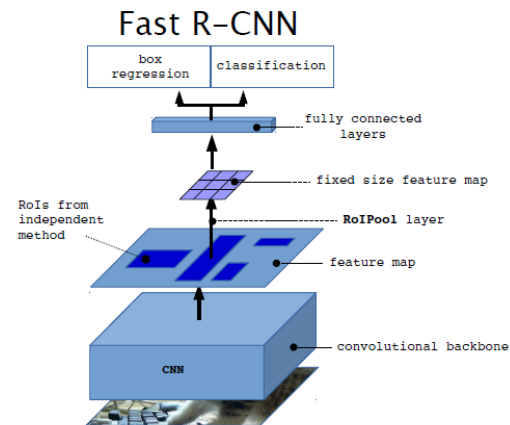
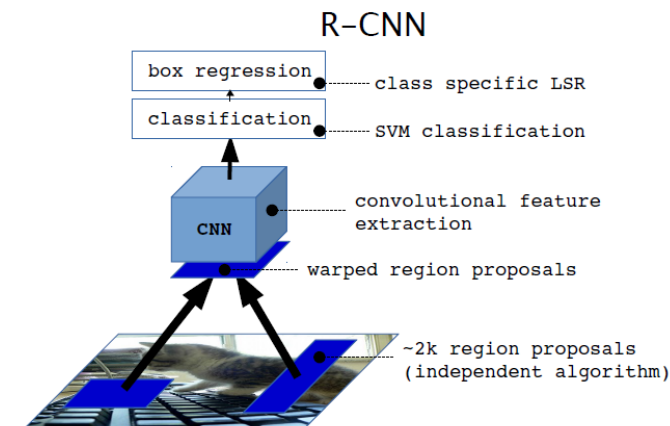


Figure 1. The **Mask R-CNN** framework for instance segmentation.



slide: C. Lim

R-CNN family



slide: C. Lim

Experimental results



Remarks: Mask R-CNN

- Instance segmentation: object detection + semantic segmentation
- Mask R-CNN extends Faster R-CNN by adding a segmentation branch (fully convolutional network, FCN)
- Based on Faster R-CNN, Mask R-CNN is simple to train and adds a small overhead to Faster R-CNN
- Running at 5fps



Outline

- Semantic segmentation
 - Fully convolutional network (FCN)
 - U-Net
- Instance segmentation
 - Mask R-CNN
- Panoptic segmentation and universal segmentation
 - Mask2Former
- Segmentation with low training data annotation costs
 - Semi-supervised learning
 - Weakly-supervised learning
 - Domain-adaptive training data synthesis



Semantic, instance, and panoptic segmentation

- An image has two main components: **things** and **stuff**
 - Countable things, such as cars, people, and apples
 - Uncountable stuff, such as road, sky, and sea
- **Semantic segmentation**: Assign each image pixel to a unique class label, including things and stuff



Image



Semantic Segmentation

Semantic, instance, and panoptic segmentation

- An image has two main components: **things** and **stuff**
 - Countable things, such as cars, people, and apples
 - Uncountable stuff, such as road, sky, and sea
- **Instance segmentation**: Assign a mask to each object instance of a thing class



Image



Instance Segmentation

Semantic, instance, and panoptic segmentation

- An image has two main components: **things** and **stuff**
 - Countable things, such as cars, people, and apples
 - Uncountable stuff, such as road, sky, and sea
- **Panoptic segmentation**: their combination
 - Perform **semantic segmentation** on **stuff**
 - Perform **instance segmentation** on **things**



Image



Instance Segmentation

Semantic, instance, and panoptic segmentation



(a) Image



(b) Semantic segmentation



(c) Instance segmentation



(d) Panoptic segmentation

Outline

- Semantic segmentation
 - Fully convolutional network (FCN)
 - U-Net
- Instance segmentation
 - Mask R-CNN
- Panoptic segmentation and universal segmentation
 - Mask2Former
- Segmentation with low training data annotation costs
 - Semi-supervised learning
 - Weakly-supervised learning
 - Domain-adaptive training data synthesis



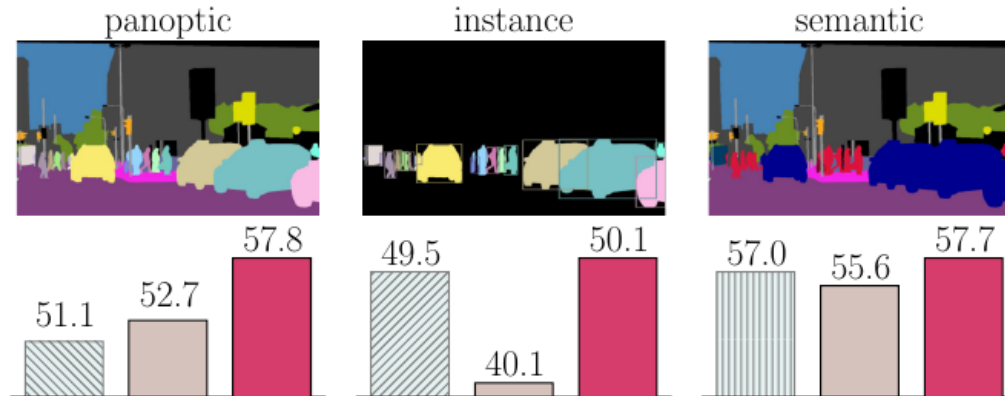
Mask2Former: Masked-attention mask transformer for universal image segmentation [Cheng et al. CVPR'22]

- Image segmentation groups pixels based on category and/or instance memberships, leading to semantic, instance, and panoptic segmentation tasks
- Specialized architectures are used for different tasks
 - FCN and UNet for semantic segmentation
 - Mask R-CNN for instance segmentation
 - DETR and Max-DeepLab [Wang et al. CVPR'21] for panoptic segmentation
 - Duplicated effort and inefficiency
- Universal architectures: They are capable of addressing all segmentation tasks with the same architectures



Contributions of Mask2Former

- Mask2Former: one model for all segmentation tasks
- Use **masked attention** in the Transformer decoder
 - Extract localized features by constraining cross-attention within predicted mask regions
- Outperforms specialized architectures



Universal architectures:

Mask2Former (ours) MaskFormer

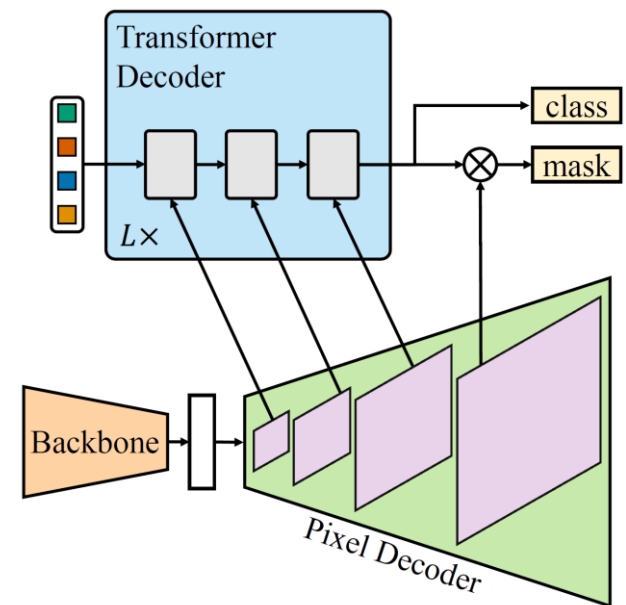
SOTA specialized architectures:

Max-DeepLab Swin-HTC++ BEiT



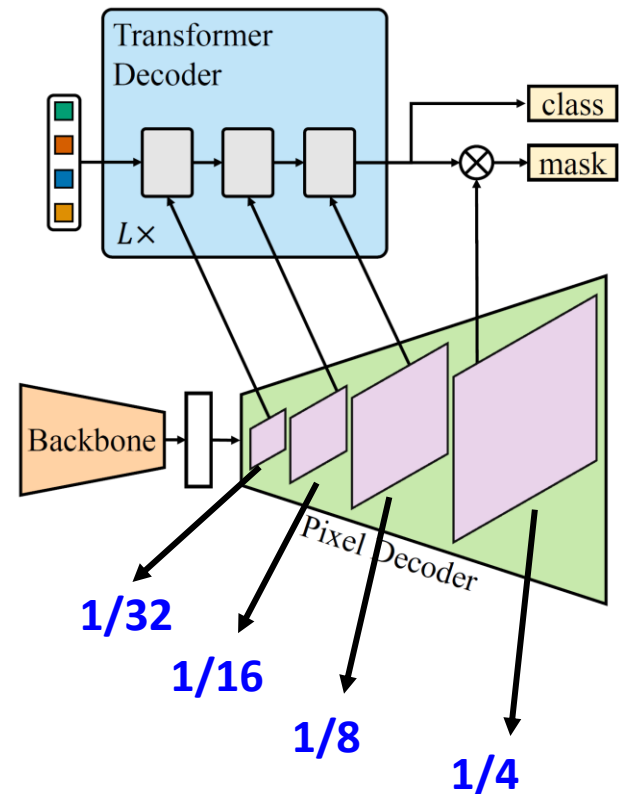
Architecture overview

- Mask2Former builds upon a meta-architecture for mask generation
- It consists of three components
 - **Backbone**: Image feature extraction
 - **Pixel decoder**: Pixel feature extraction
 - **Transformer decoder**:
 - ◆ Input: Mask queries
 - ◆ **Masked attention** for mask feature aggregation
 - ◆ Output: Masks and their classes



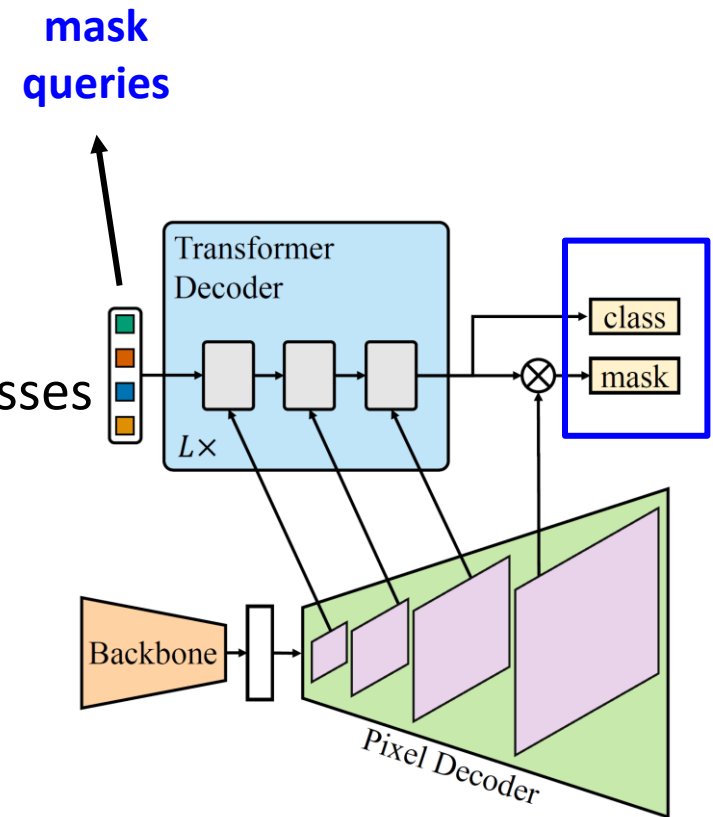
Architecture: Backbone and pixel decoder

- **Backbone**: Extract low-resolution features from an image
 - ResNet or Swin Transformer
- **Pixel decoder**: Gradually upsample low-resolution features to generate high-resolution per-pixel embeddings
 - FPN or MSDeformAttn
 - Four-scale feature maps with resolutions $1/32$, $1/16$, $1/8$, and $1/4$
 - The first three scales are used for mask feature aggregation
 - The last scale serves as per-pixel embedding



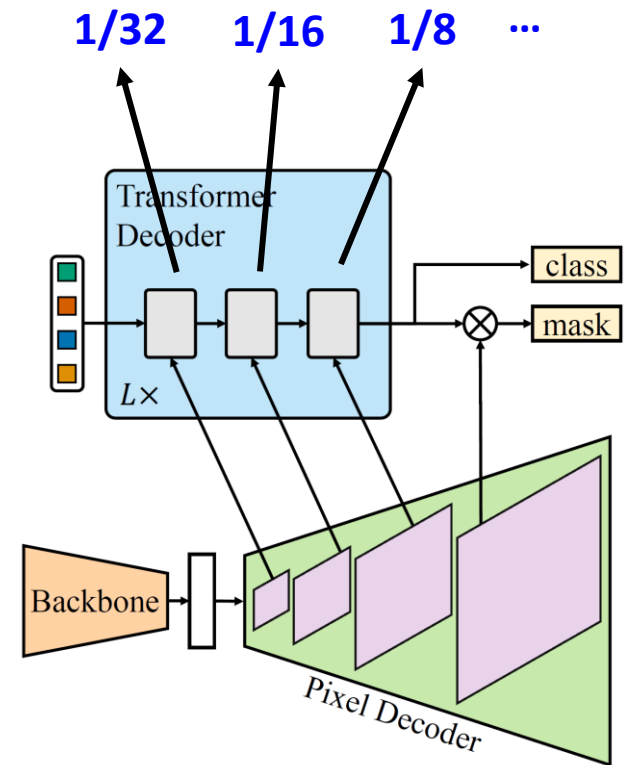
Architecture: Transformer decoder

- Input: Mask queries
- Output: Masks and their classes
- Recall DETR
 - Input: object queries
 - Output: Bounding boxes and their classes



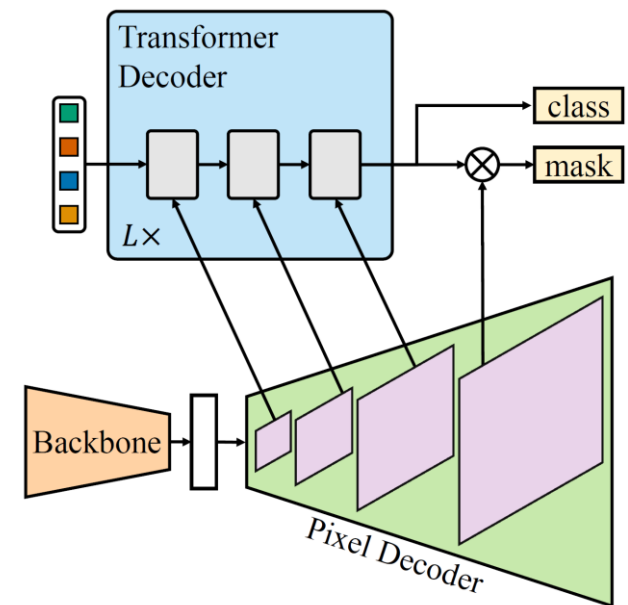
Architecture: Transformer decoder

- Input: Mask queries
- Output: Masks and their classes
- Decoder layers perform **mask generation** and **feature aggregation**
- Successive feature maps from the pixel decoder's are fed into the transformer decoder in a **round-robin** fashion
 - One scale at each layer
- Each decoder layer performs **masked attention** for mask generation and feature aggregation



Architecture: Transformer decoder

- Transformer decoder carries out mask generation and feature aggregation
- For **mask generation**, the transformer decoder takes feature maps from the previous layer and learns masks in form of $\mathbf{M}_{l-1} \in \{0, 1\}^{N \times H_l W_l}$, the binarized mask prediction of the previous $(l-1)$ -th Transformer decoder layer

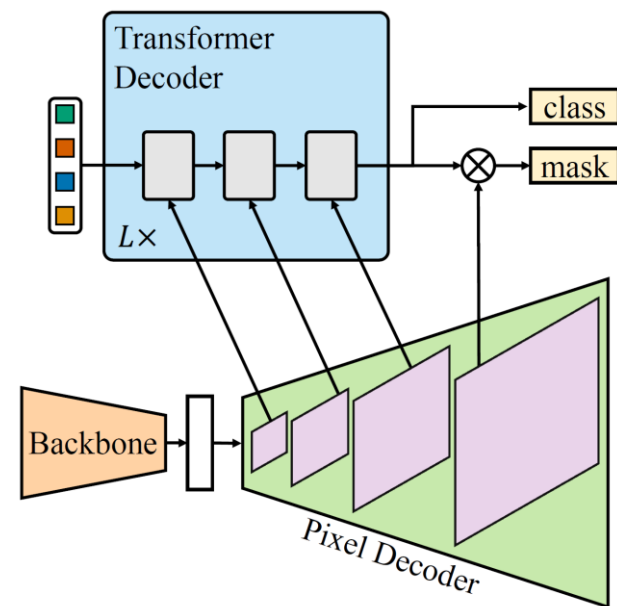


Architecture: Transformer decoder

- For **feature aggregation**, the **masked attention** is performed
 - Hypothesis: Local features are enough to update query features and context information can be gathered through self-attention
 - Masked attention considers only the foreground region of the predicted mask for each query

$$\mathbf{X}_l = \text{softmax}(\mathcal{M}_{l-1} + \mathbf{Q}_l \mathbf{K}_l^T) \mathbf{V}_l + \mathbf{X}_{l-1}$$

$$\mathcal{M}_{l-1}(x, y) = \begin{cases} 0 & \text{if } \mathbf{M}_{l-1}(x, y) = 1 \\ -\infty & \text{otherwise} \end{cases}$$



Experimental results: Comparison with SoTAs

Panoptic segmentation on COCO panoptic val2017 with 133 categories

method	backbone	query type	epochs	PQ	PQ Th	PQ St	AP _{pan} Th	mIoU _{pan}	#params.	FLOPs	fps
DETR [5]	R50	100 queries	500+25	43.4	48.2	36.3	31.1	-	-	-	-
MaskFormer [14]	R50	100 queries	300	46.5	51.0	39.8	33.0	57.8	45M	181G	17.6
Mask2Former (ours)	R50	100 queries	50	51.9	57.7	43.0	41.7	61.7	44M	226G	8.6
DETR [5]	R101	100 queries	500+25	45.1	50.5	37.0	33.0	-	-	-	-
MaskFormer [14]	R101	100 queries	300	47.6	52.5	40.3	34.1	59.3	64M	248G	14.0
Mask2Former (ours)	R101	100 queries	50	52.6	58.5	43.7	42.6	62.4	63M	293G	7.2
Max-DeepLab [52]	Max-L	128 queries	216	51.1	57.0	42.2	-	-	451M	3692G	-
MaskFormer [14]	Swin-L [†]	100 queries	300	52.7	58.5	44.0	40.1	64.8	212M	792G	5.2
K-Net [62]	Swin-L [†]	100 queries	36	54.6	60.2	46.0	-	-	-	-	-
Mask2Former (ours)	Swin-L [†]	200 queries	100	57.8	64.2	48.1	48.6	67.4	216M	868G	4.0

Instance segmentation on COCO val2017 with 80 categories

method	backbone	query type	epochs	AP	AP ^S	AP ^M	AP ^L	AP ^{boundary}	#params.	FLOPs	fps
MaskFormer [14]	R50	100 queries	300	34.0	16.4	37.8	54.2	23.0	45M	181G	19.2
Mask R-CNN [24]	R50	dense anchors	36	37.2	18.6	39.5	53.3	23.1	44M	201G	15.2
Mask R-CNN [18, 23, 24]	R50	dense anchors	400	42.5	23.8	45.0	60.0	28.0	46M	358G	10.3
Mask2Former (ours)	R50	100 queries	50	43.7	23.4	47.2	64.8	30.6	44M	226G	9.7
Mask R-CNN [24]	R101	dense anchors	36	38.6	19.5	41.3	55.3	24.5	63M	266G	10.8
Mask R-CNN [18, 23, 24]	R101	dense anchors	400	43.7	24.6	46.4	61.8	29.1	65M	423G	8.6
Mask2Former (ours)	R101	100 queries	50	44.2	23.8	47.7	66.7	31.1	63M	293G	7.8
QueryInst [20]	Swin-L [†]	300 queries	50	48.9	30.8	52.6	68.3	33.5	-	-	3.3
Swin-HTC++ [6, 36]	Swin-L [†]	dense anchors	72	49.5	31.0	52.4	67.2	34.1	284M	1470G	-
Mask2Former (ours)	Swin-L [†]	200 queries	100	50.1	29.9	53.9	72.1	36.2	216M	868G	4.0



Experimental results: Comparison with SoTAs

Semantic segmentation on ADE20K val with 150 categories

method	backbone	crop size	mIoU (s.s.)	mIoU (m.s.)
MaskFormer [14]	R50	512	44.5	46.7
Mask2Former (ours)	R50	512	47.2	49.2
Swin-UperNet [36, 58]	Swin-T	512	-	46.1
MaskFormer [14]	Swin-T	512	46.7	48.8
Mask2Former (ours)	Swin-T	512	47.7	49.6
MaskFormer [14]	Swin-L [†]	640	54.1	55.6
FaPN-MaskFormer [14, 39]	Swin-L-FaPN [†]	640	55.2	56.7
BEiT-UperNet [2, 58]	BEiT-L [†]	640	-	57.0
Mask2Former (ours)	Swin-L [†]	640	56.1	57.3
	Swin-L-FaPN [†]	640	56.4	57.7



Visualization: Instance segmentation



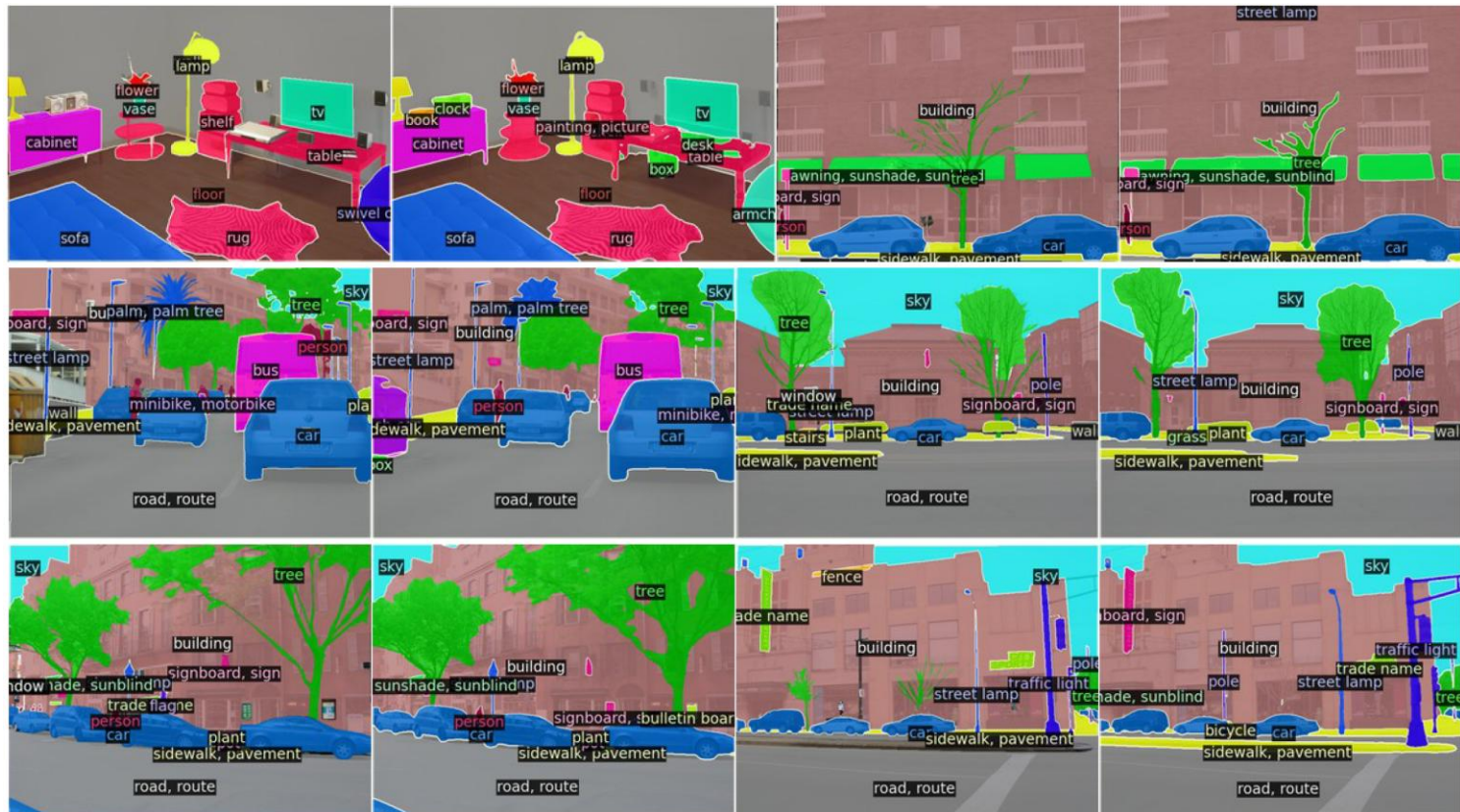
- Left: Ground truth; Right: Prediction by Mask2Former

Visualization: Panoptic segmentation



- Left: Ground truth; Right: Prediction by Mask2Former

Visualization: Semantic segmentation



- Left: Ground truth; Right: Prediction by Mask2Former

References

- FCN

- J. Long, E. Shelhamer, and T. Darrell: Fully convolutional networks for semantic segmentation, in CVPR 2015.

- U-Net

- O. Ronneberger, P. Fischer, and T. Brox: U-Net: Convolutional networks for biomedical image segmentation, in MICCAI, 2015.

- Mask R-CNN

- K. He, G. Gkioxari, P. Dollár, and R. Girshick: Mask R-CNN, in ICCV 2017.

- Mask2Former

- B. Cheng, I. Misra, A. Schwing, A. Kirillov, and R. Girdhar: Masked-attention mask transformer for universal image segmentation, in CVPR 2022.



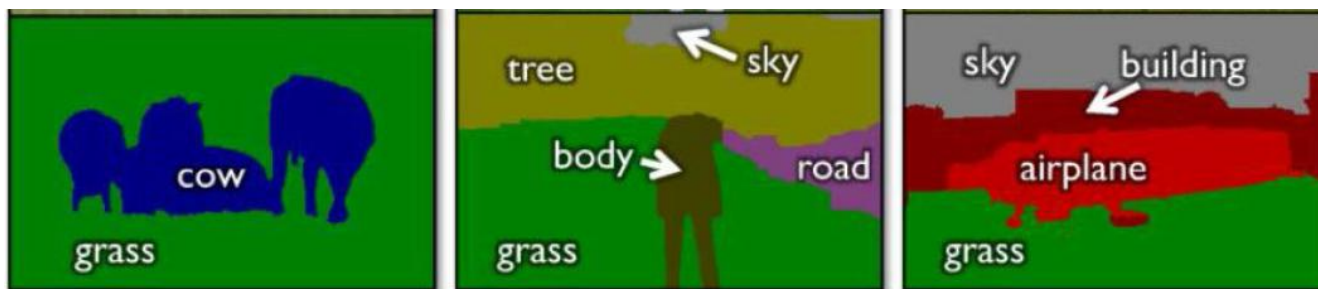
Outline

- Semantic segmentation
 - Fully convolutional network (FCN)
 - U-Net
- Instance segmentation
 - Mask R-CNN
- Panoptic segmentation and universal segmentation
 - Mask2Former
- Segmentation with low training data annotation costs
 - Semi-supervised learning
 - Weakly-supervised learning
 - Domain-adaptive training data synthesis



Motivation for segmentation with low annotation costs

- Deep learning relies on a vast amount of training data
- This issue becomes worse for object segmentation
- Training data with **pixel-wise annotations** are required



- **Motivation** is threefold:
 - 1. Segmentation is important
 - 2. Deep learning is data hungry
 - 3. Pixel-wise annotation is required for segmentation

Some of my research outcomes on this topic

- **Semi-supervised** semantic segmentation
 - [Hung et al. BMVC'18]
- **Weakly-supervised** instance segmentation
 - [Hsu et al. NeurIPS'19]
- Segmentation with **domain-adaptive training data synthesis**
 - [Chen et al. CVPR'19]



Some of my research outcomes on this topic

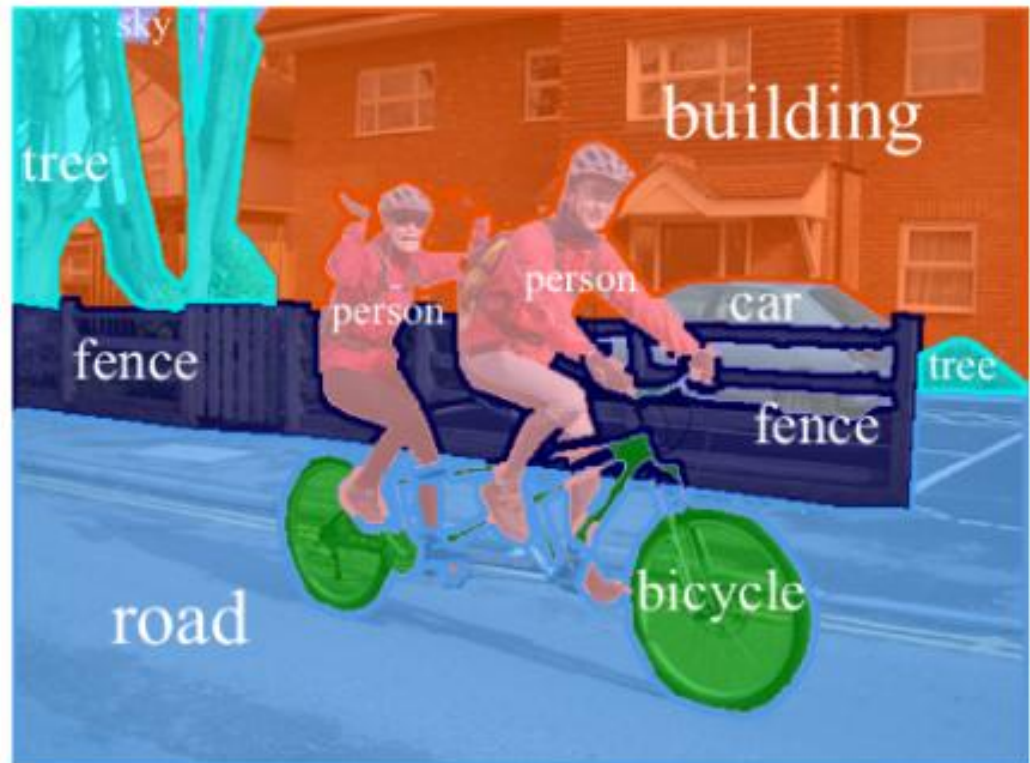
- Semi-supervised semantic segmentation
- Weakly-supervised instance segmentation
- Segmentation with domain-adaptive training data synthesis



Semantic segmentation

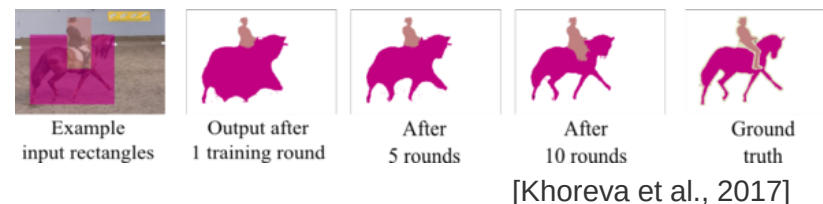
[Hung et al. BMVC'18]

- Label each pixel with a semantic class
- Applications
 - Autonomous driving
 - Image editing
 - Medical care



Pixel-level annotations are expensive

- Solution 1: Reduce annotation **effort** per image
 - Change annotation form: image labels, bounding boxes, points
 - **Weakly**-supervised learning

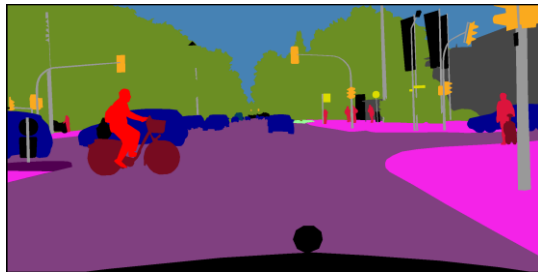


- Solution 2: Reduce annotation **amount**
 - Only label partial images in the dataset
 - **Semi**-supervised learning

The focus of this work



Data for semi-supervised learning



Small amount of labeled data



Large amount of unlabeled data

How do we exploit these data?



Motivation: Exploit structured context

[Hung et al. BMVC'18]

Labeled Data



Image



Ground truth

Labeled/Unlabeled Data



Image



Model Prediction

Can we push them
to have similar
structure contexts?

Apply adversarial
learning on the
output space.

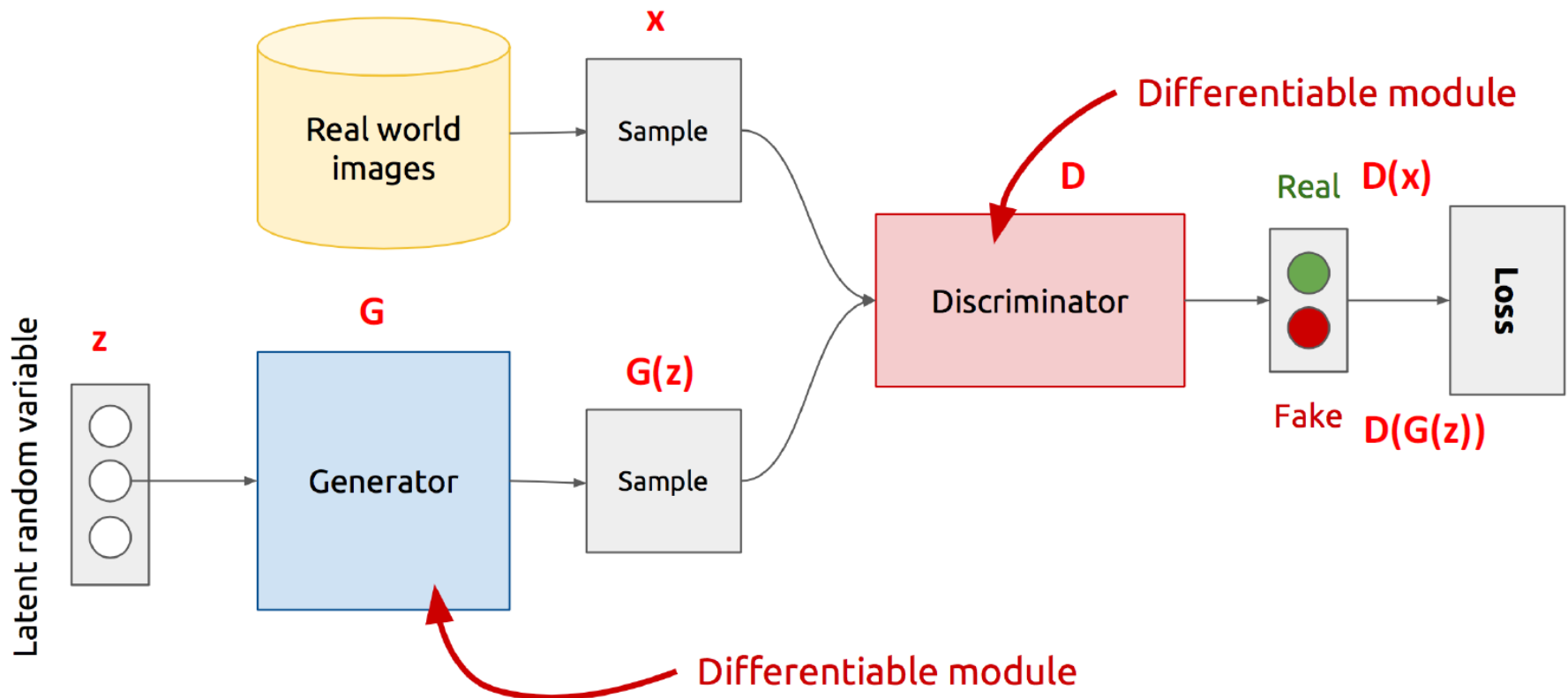
Adversarial learning

[Goodfellow et al. NeurIPS'14]

- Generative Adversarial Networks (GANs)
 - Train two different networks: **Generator** and **Discriminator**
 - Given a set of real images during training
 - **Generator** tries to produce realistic-looking images and fool the discriminator
 - **Discriminator** tries to figure out if an image comes from the training set or generator
 - Train **Generator** and **Discriminator** against each other
 - Repeat this and we get better **Generator** and **Discriminator**



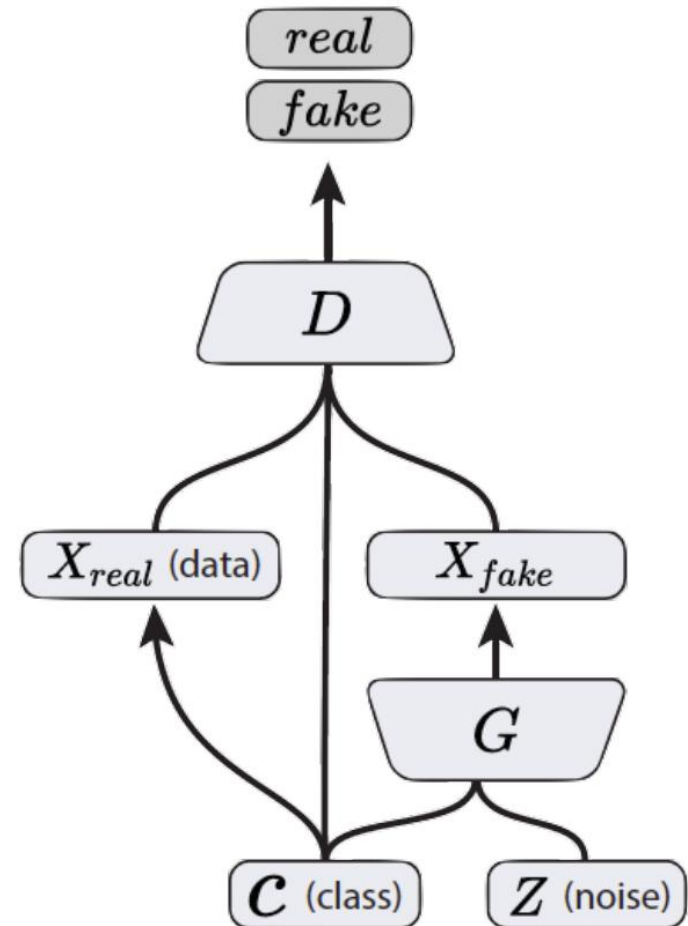
GAN's architecture



<https://www.slideshare.net/xavigiro/deep-learning-for-computer-vision-generative-models-and-adversarial-training-upc-2016>

Conditional GAN

- Modification to GAN to condition the model on **additional information** for better multi-modal learning when we have explicit **supervision** available

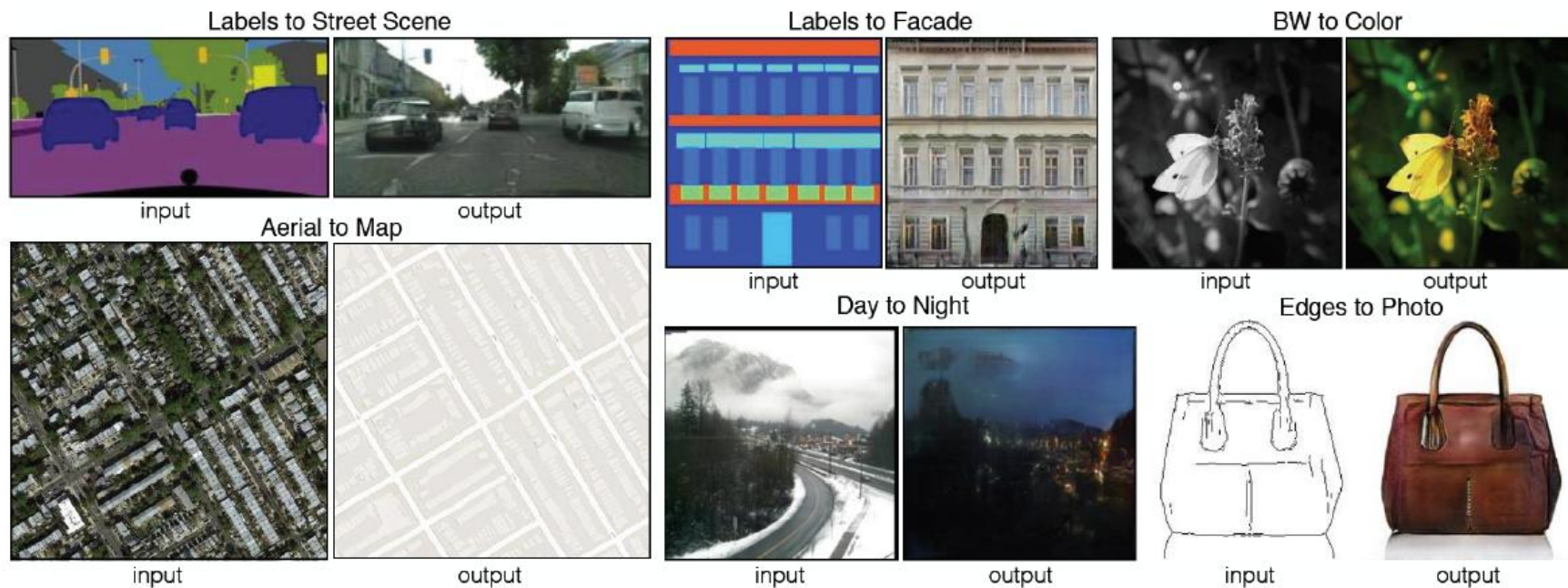


Conditional GAN
(Mirza & Osindero, 2014)

Image-to-image translation

[Isola et al., CVPR'17]

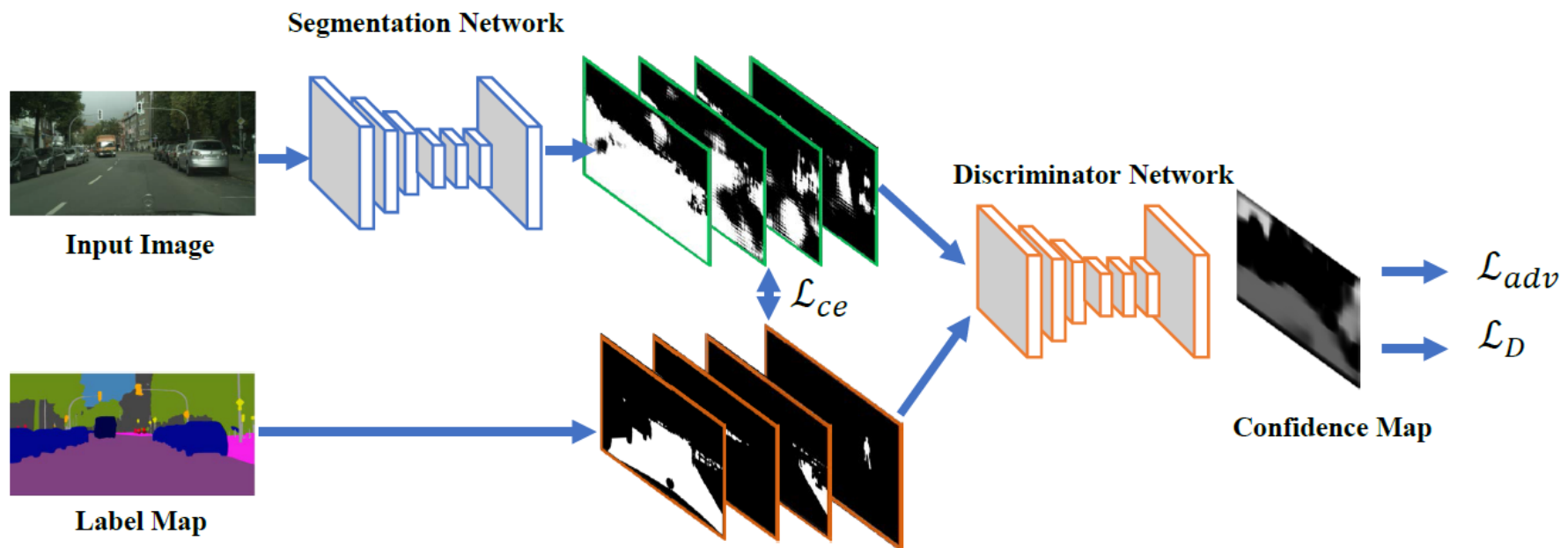
- A new image can be generated by conditioning on **another image**, instead of the class label



Semi-supervised semantic segmentation

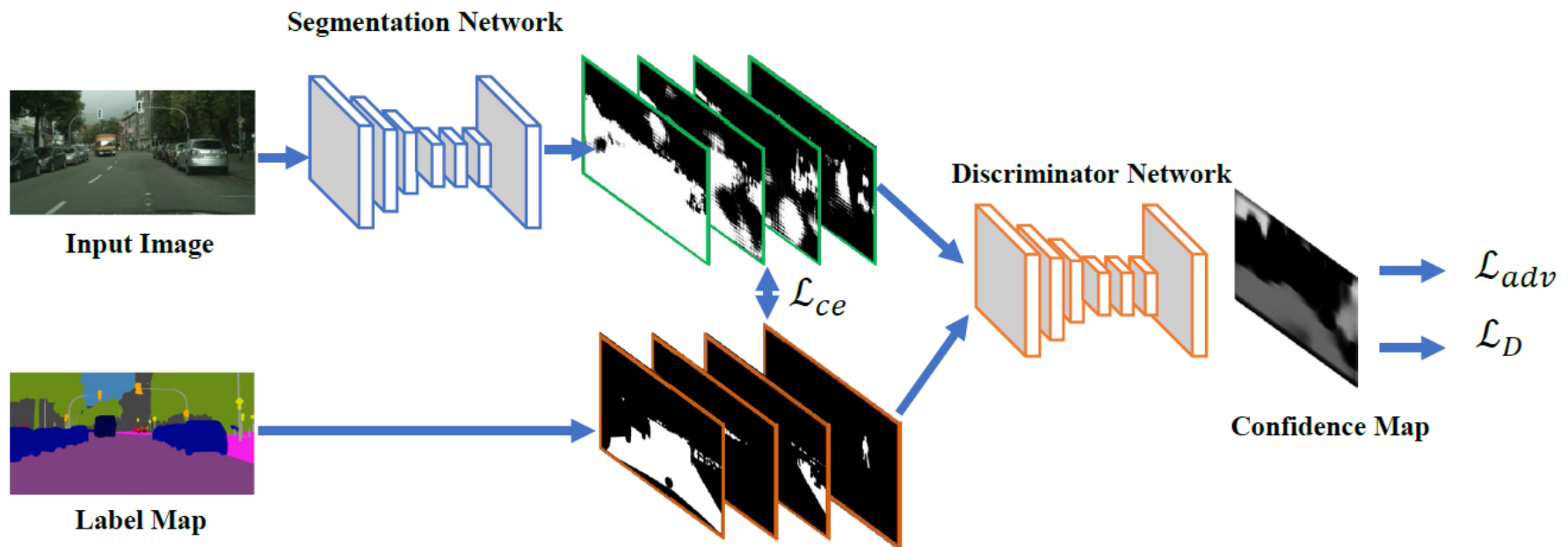
[Hung et al. BMVC'18]

- **Generator** produces **segmentation maps** instead of images
- **Fully convolutional discriminator** for pixel-wise prediction



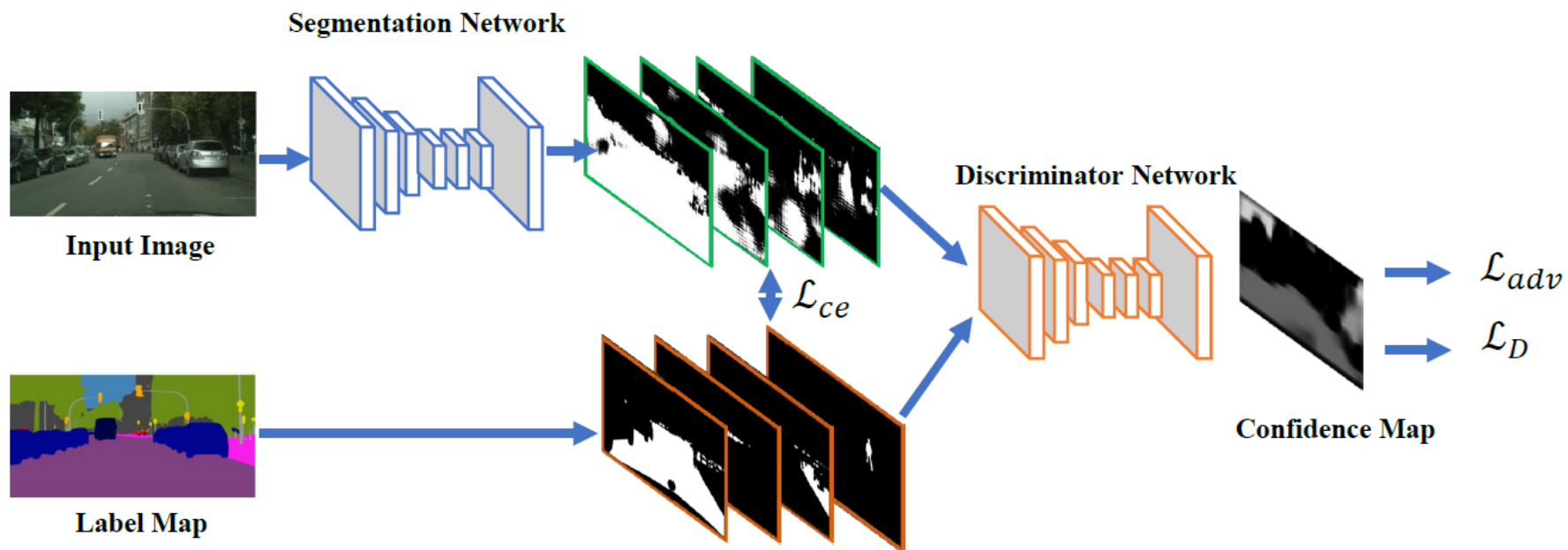
Generator learning

- **Generator** produces **segmentation maps** instead of images
- $\mathcal{L}_{ce}$: Supervised learning with the labeled data
- $\mathcal{L}_{adv}$: Adversarial learning for the generator, fooling the discriminator



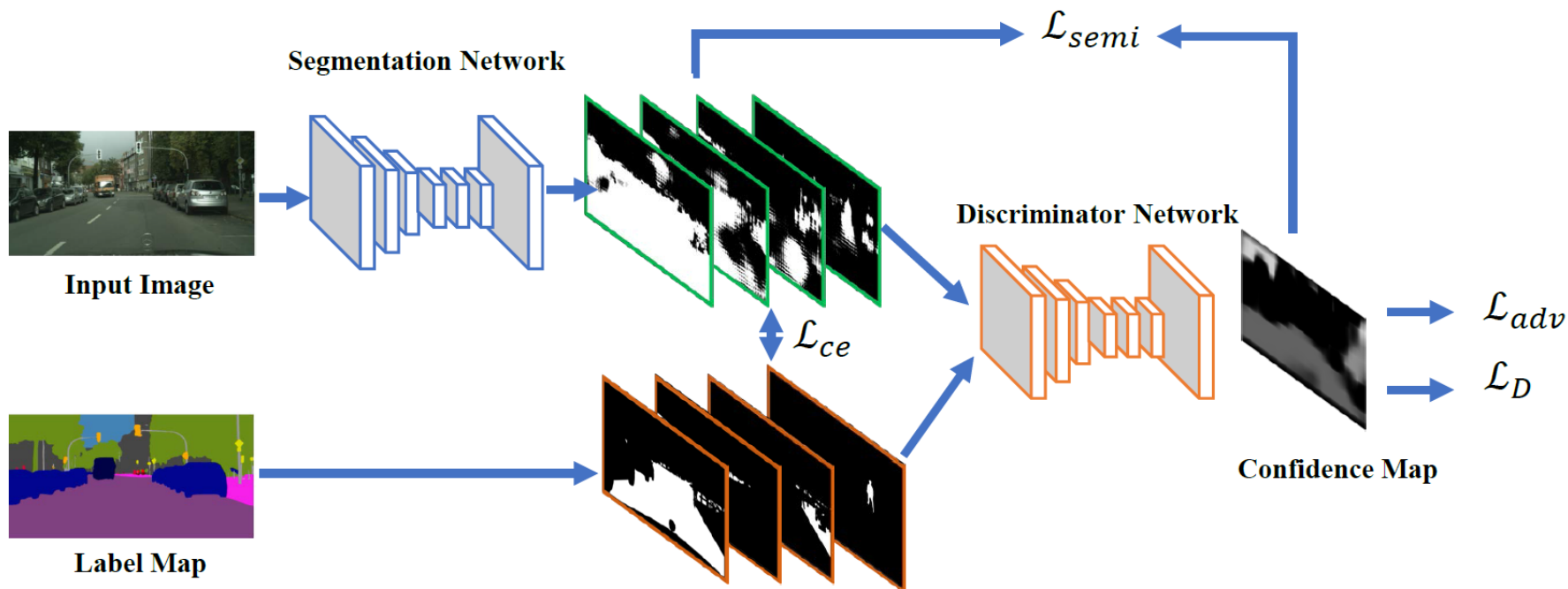
Discriminator learning

- Fully convolutional discriminator for pixel-wise prediction
- $\mathcal{L}_D$: Adversarial learning for the discriminator, separating the ground-truth segmentation maps and the generated segmentation maps



Semi-supervised learning

- Enable semi-supervised learning
 - Infer high-confidence regions of the generated segmentation maps
 - Obtain the pseudo ground truth for these regions
 - Use these regions with pseudo ground truth to learn the generator



Experimental setting

- Dataset
 - Cityscapes, PASCAL VOC 2012
- Segmentation network
 - DeepLab-v2: Resnet-101, single scale, no CRF
- Discriminator network
 - DCGAN: 4 conv layers + Leaky ReLU, fully convolutional
- Implementation framework
 - PyTorch



Comparisons with SoTA on Pascal VOC

- Labeled data: Pascal VOC; Unlabeled data: BDS

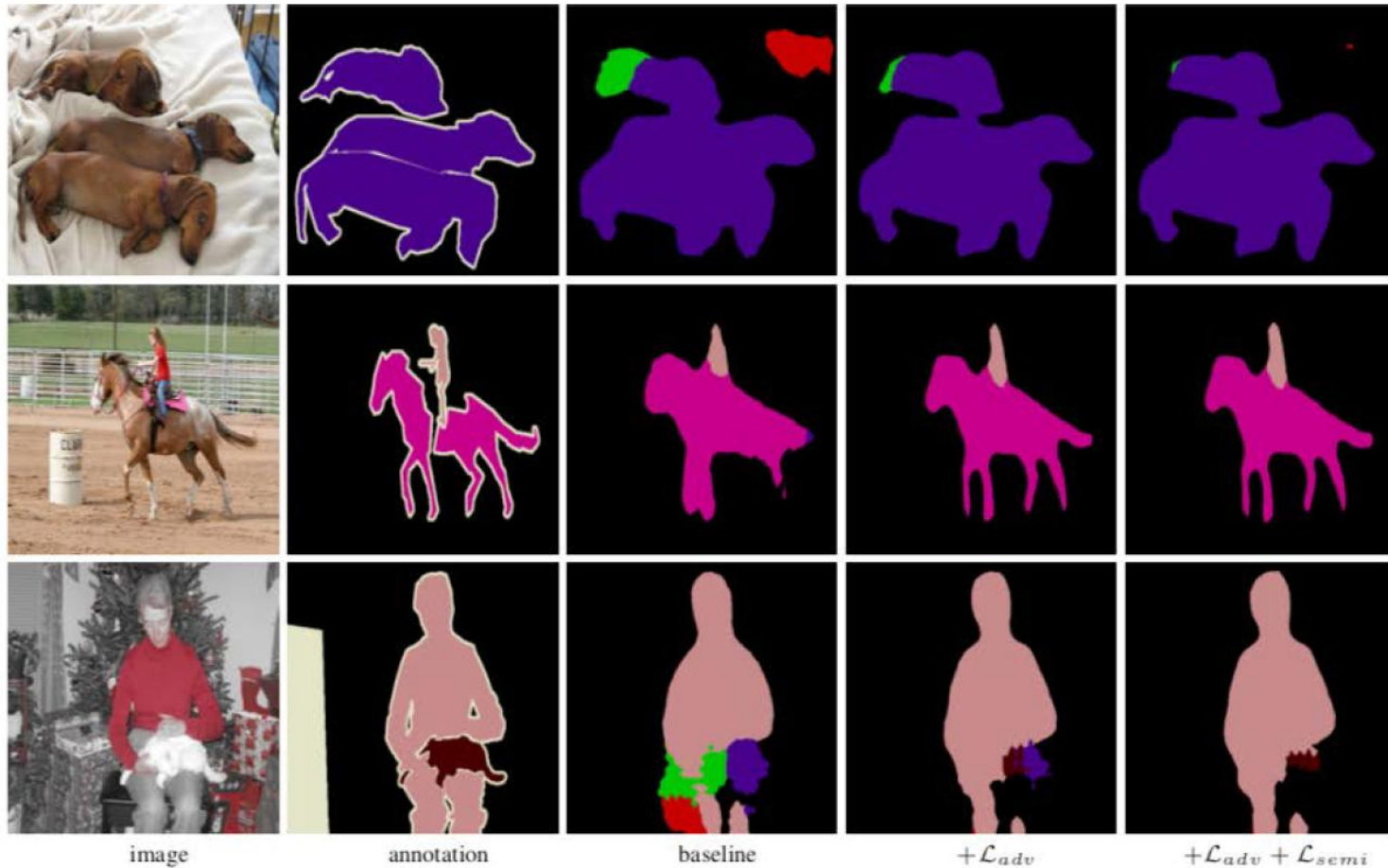
	Data Amount	Fully-supervised	Semi-supervised
[1]	Full	62.5	64.6
[2]	Full	59.5	64.1
ours	Full	66.3	68.4
[2]	30%	38.9	42.2
ours	30%	57.4	60.6

[1] George Papandreou, Liang-Chieh Chen, Kevin Murphy, and Alan L Yuille. Weakly-and semi-supervised learning of a dcnn for semantic image segmentation. In *ICCV*, 2015

[2] Nasim Souly, Concetto Spampinato, and Mubarak Shah. Semi and weakly supervised semantic segmentation using generative adversarial network. In *ICCV*, 2017.



Qualitative results



Code Available at:

<https://github.com/hfslyc/AdvSemiSeg>



Some of my research outcomes on this topic

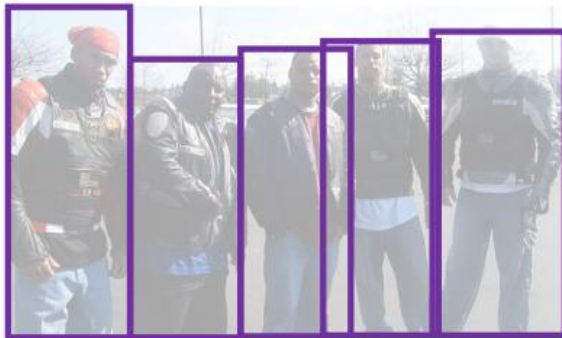
- Semi-supervised semantic segmentation
- Weakly-supervised instance segmentation
- Segmentation with domain-adaptive training data synthesis



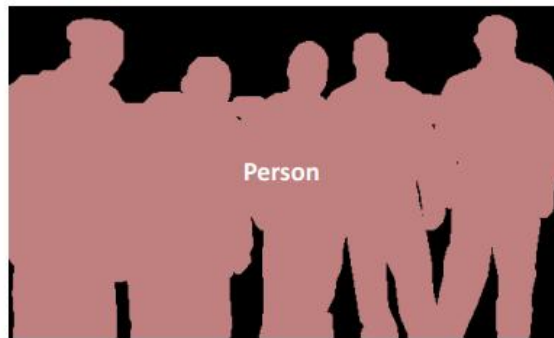
Weakly supervised instance segmentation

[Hsu et al. NeurIPS'19]

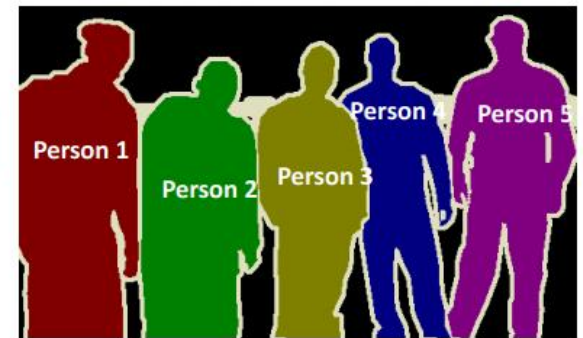
- Instance segmentation
= object detection + semantic segmentation
- Jointly estimates classes and segmentation masks of individual objects



Object Detection



Semantic Segmentation



Instance Segmentation

Motivation

- Motivation: Reduce the annotation cost
- What is annotation cost?

➤ Total labeling time of the training set

	IL	IL+C	Full	BB
Cost (s/image)	20	22.22	239.7	38.1

Table 1: Average annotation cost per image when using different types of supervision.

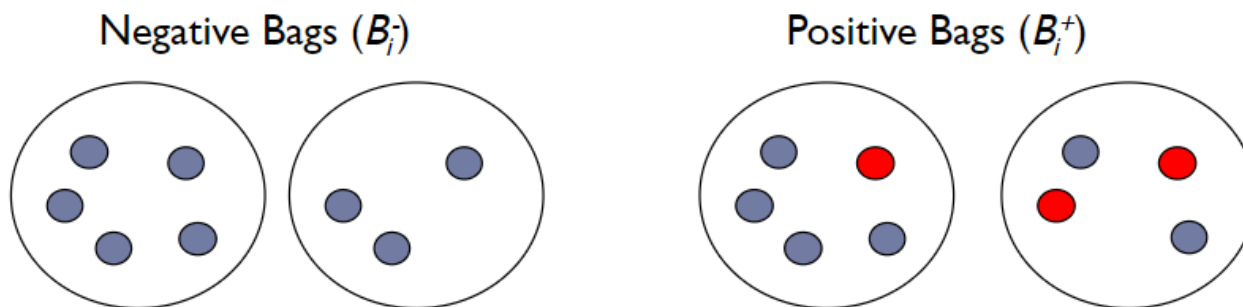
[Bellver et al. arXiv'19]

- **IL**: image-level annotations
- **IL+C**: image-level + counts annotations
- **Full**: fully supervised instance segmentation annotations
- **BB**: bounding box annotations
- Goal: Training an instance segmentation model with **box-level annotations**

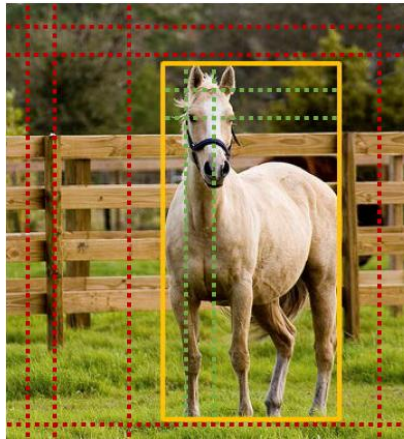


Multiple instance learning (MIL)

- In MIL, instead of giving the learner labels for the individual examples, the trainer only labels collections of examples, which are called bags
 - **Bag**: a collection of examples
 - **Example**: a data instance
- A bag is labeled **positive** if there is at least one positive example in it
- It is labeled **negative** if all the examples in it are negative



MIL for bounding box annotations: Observation



Positive Bags



Negative Bags



- A bounding box:
 - The enclosed instance must touch the four sides of its bounding box
 - No overlap between the instance and the region outside its bounding box

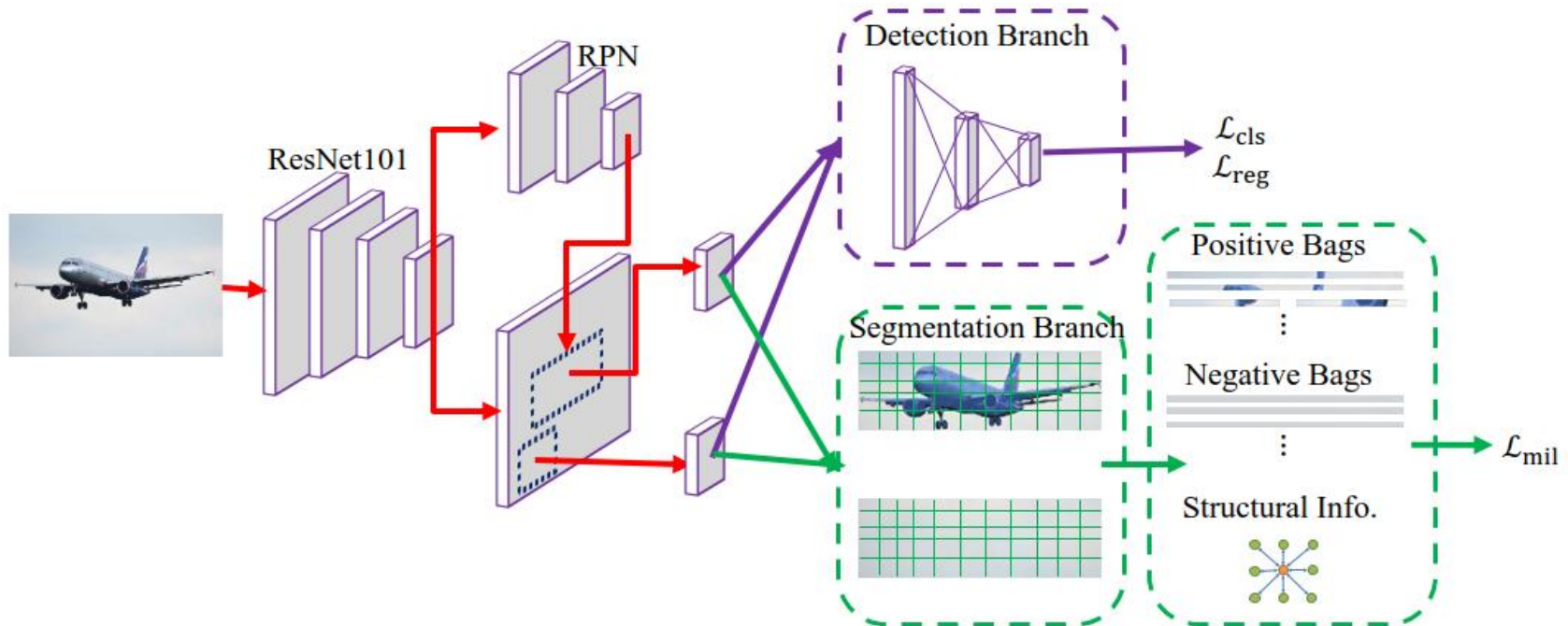
MIL for bounding box annotations: Formulation



- **Positive bag**: a sweeping line inside the bounding box
 - ✓ At least one pixel on this line belongs to the enclosed object
- **Negative bag**: a sweeping line having no overlap with any bounding box
 - ✓ No object pixel is present on this line

Network architecture

- Our model is developed based on Mask R-CNN
- Use the proposed MIL-based loss to learn the segmentation branch



Objective function for learning Mask R-CNN

- Mask R-CNN: backbone network optimized with the three losses

$$\mathcal{L}(\mathbf{w}) = \mathcal{L}_{\text{cls}}(\mathbf{w}) + \mathcal{L}_{\text{reg}}(\mathbf{w}) + \mathcal{L}_{\text{mil}}(\mathbf{w}),$$

- $\mathcal{L}_{\text{cls}}$ and $\mathcal{L}_{\text{reg}}$: optimized with the given the box-level annotation
- $\mathcal{L}_{\text{mil}}$: MIL loss, a new loss which enables Mask R-CNN to be trained **with box-level annotations**, instead of pixel-level ones



The proposed MIL loss $\mathcal{L}_{mil}$

- MIL loss $\mathcal{L}_{mil}$:

$$\mathcal{L}_{mil}(S; \hat{B}^+, \hat{B}^-) = \psi(S; \hat{B}^+, \hat{B}^-) + \phi(S)$$

- S : instance score map predicted by the segmentation branch
- $\hat{B}^+$ and $\hat{B}^-$: positive and negative bags for each given bounding box b
- Unary term ψ
 - Enforcing the MIL constraints
- Pairwise term ϕ
 - Maintaining integrity of the object



The proposed MIL loss $\mathcal{L}_{mil}$

- Unary term ψ :

$$\psi(S; \hat{B}^+, \hat{B}^-) = \sum_{\hat{b} \in \hat{B}^+} -\log P(\hat{b}) + \sum_{\hat{b} \in \hat{B}^-} -\log (1 - P(\hat{b}))$$

➤ $P(\hat{b})$: probability of the bag $\hat{b}$

- Pairwise term ϕ :

$$\phi(S) = \sum_{(\mathbf{p}, \mathbf{p}') \in \mathcal{E}} \|S(\mathbf{p}) - S(\mathbf{p}')\|^2$$

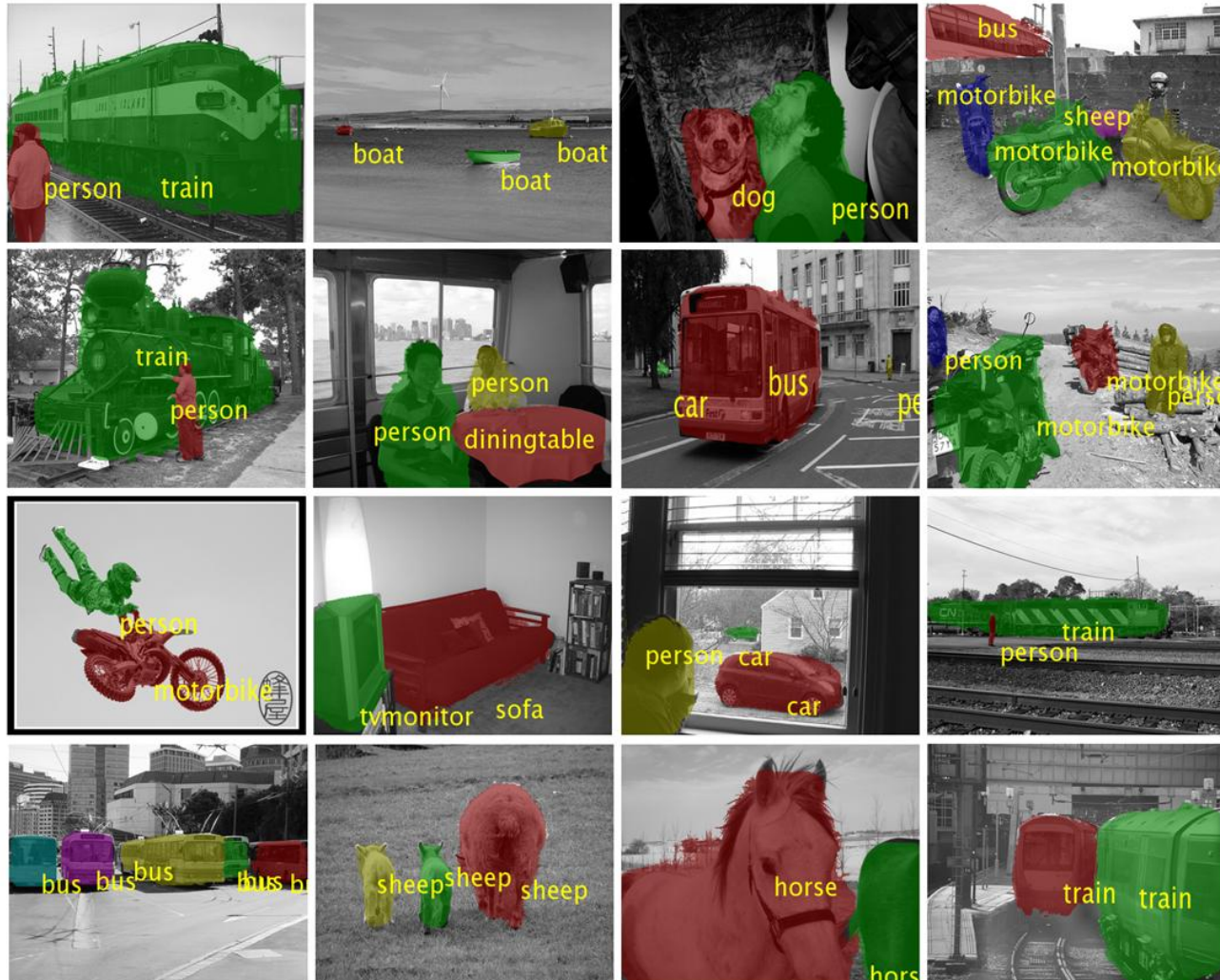
➤ $\mathcal{E}$: the set containing all neighboring pixel pairs



Experimental results

Code available at:

https://github.com/chengchunhsu/WSIS_BBTP



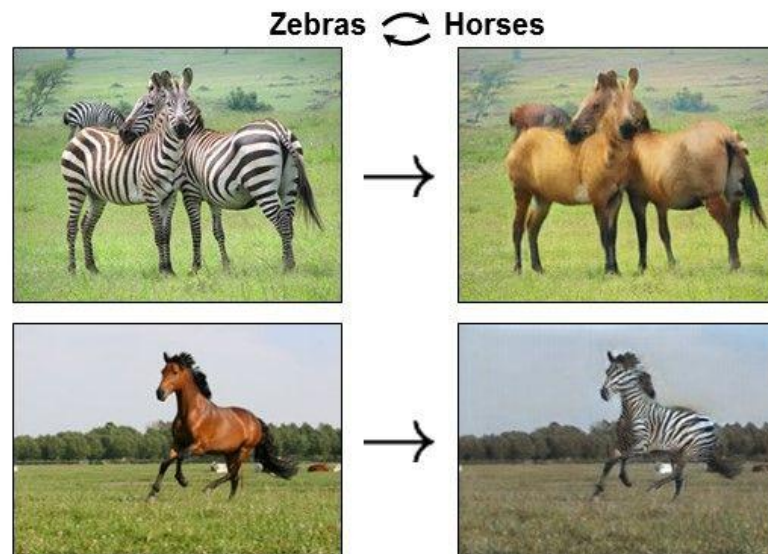
Some of my research outcomes on this topic

- Semi-supervised semantic segmentation
- Weakly-supervised instance segmentation
- Segmentation with domain-adaptive training data synthesis



High-quality segmentation data generation

- Image-to-image translation, such as CycleGAN:



- Suppose there are a lot of training data in domain A
- If we can translate data from domain A to domain B, we will have sufficient training data to learn a model in domain B



GTA5 dataset

- The GTA5 dataset has 24,966 **synthetic** images with **pixel-wise annotations**
 - Images and semantic maps are from computer games



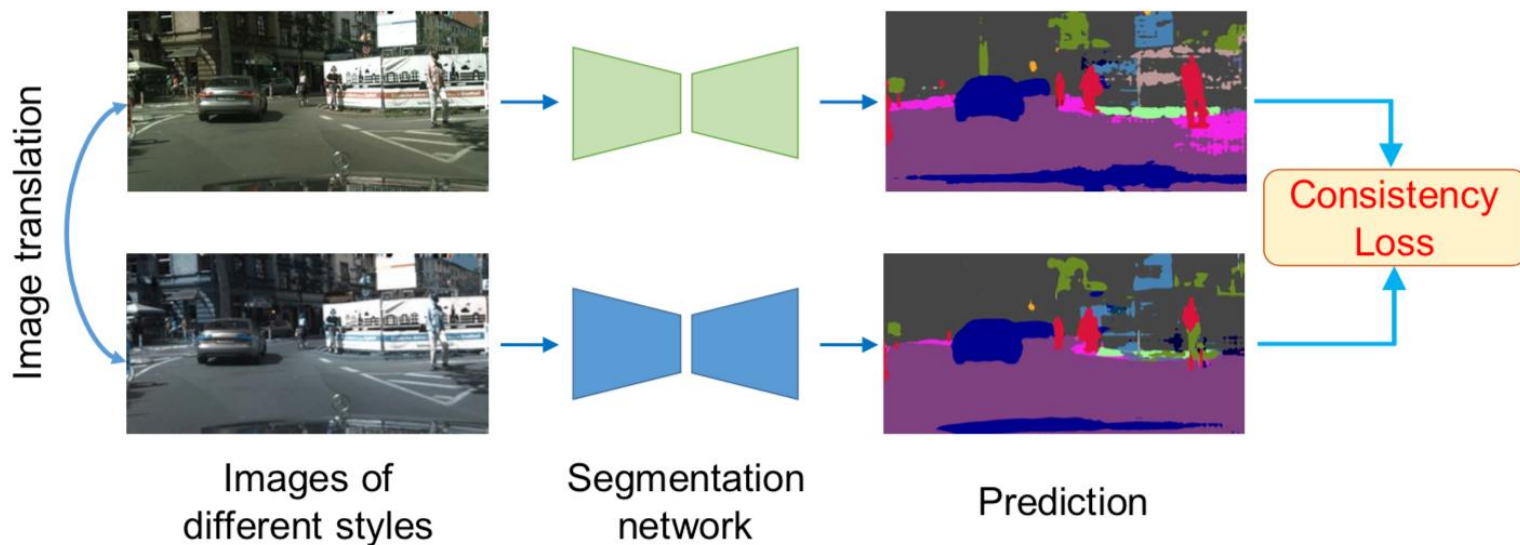
Photo: Richter et al.



Our idea

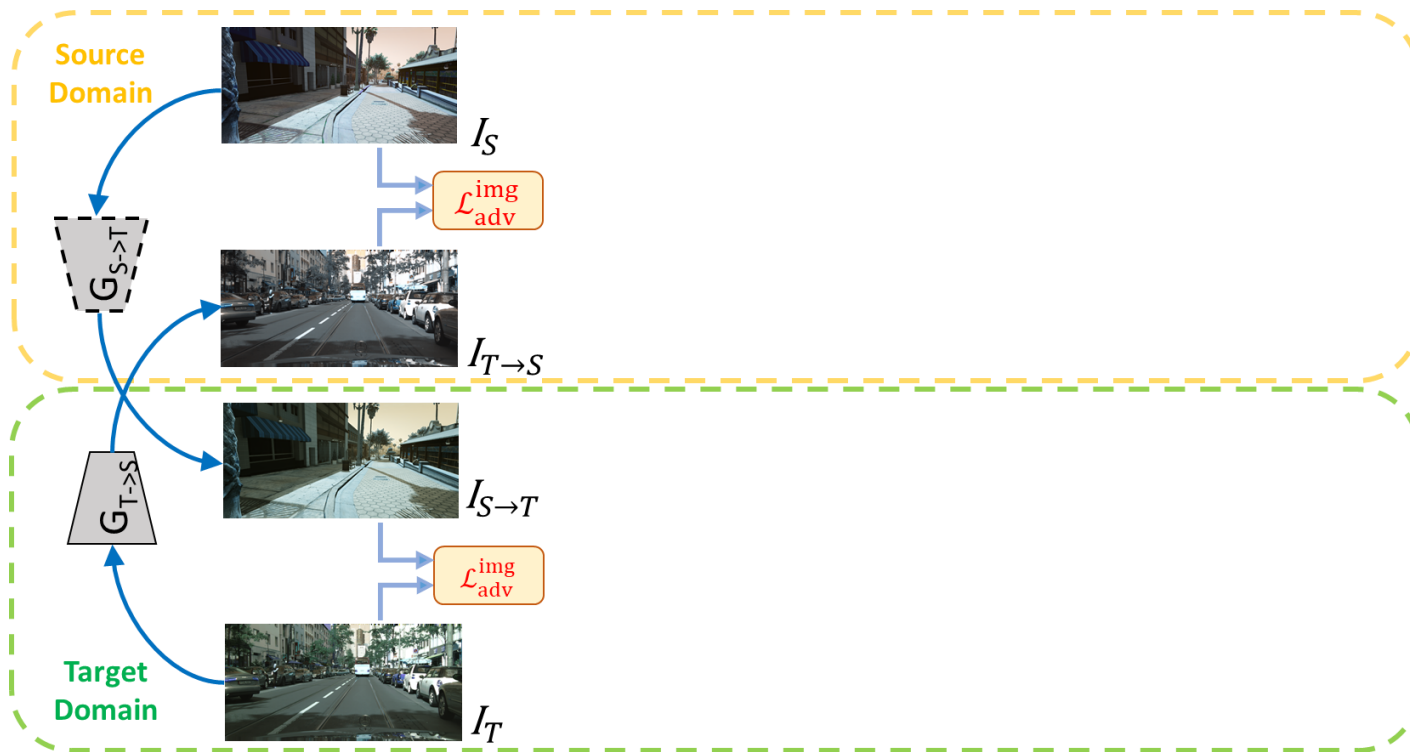
[Chen et al. CVPR'19]

- Challenge: Domain gap between the source and target datasets (i.e., domain shift)
- Images in different domains may have different styles
- Task prediction should be exactly the same



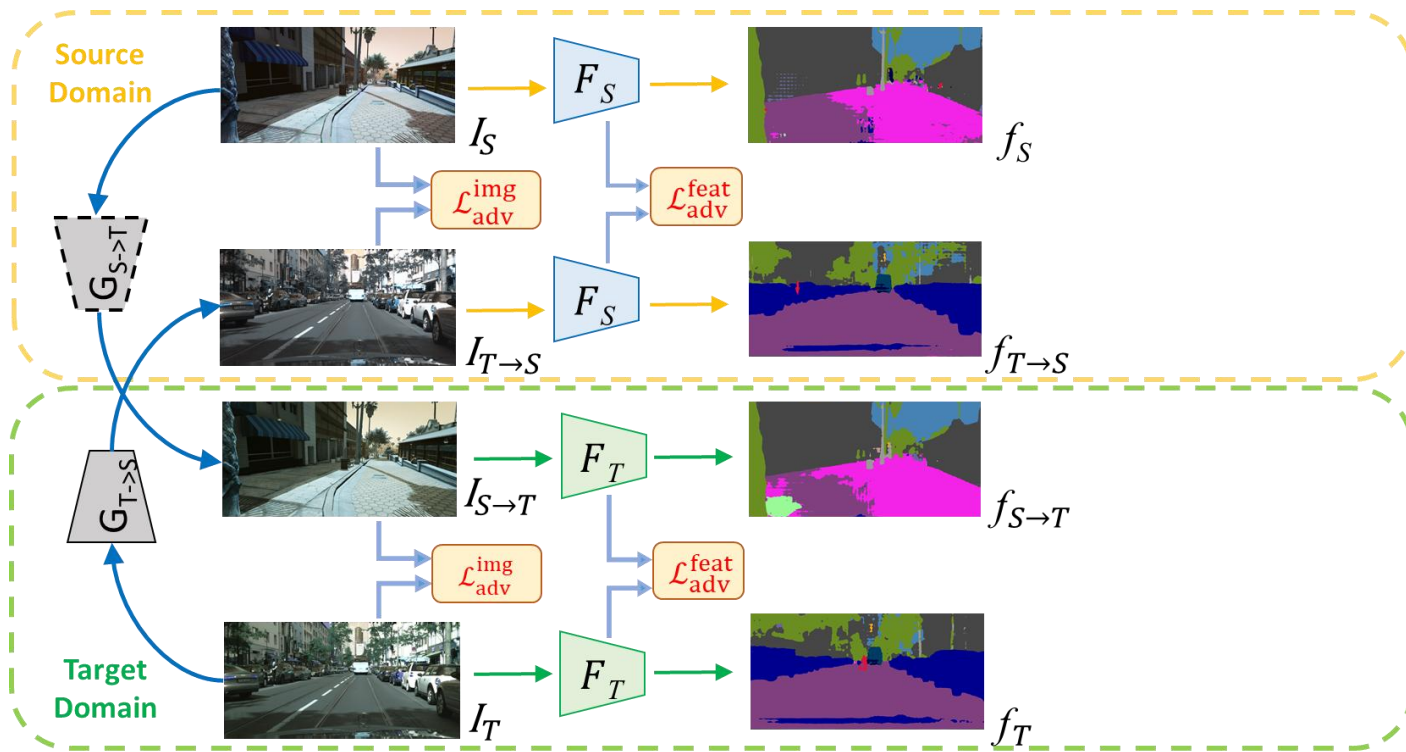
The proposed CrDoCo

- CycleGAN for the translation networks
- Image-level adversarial learning



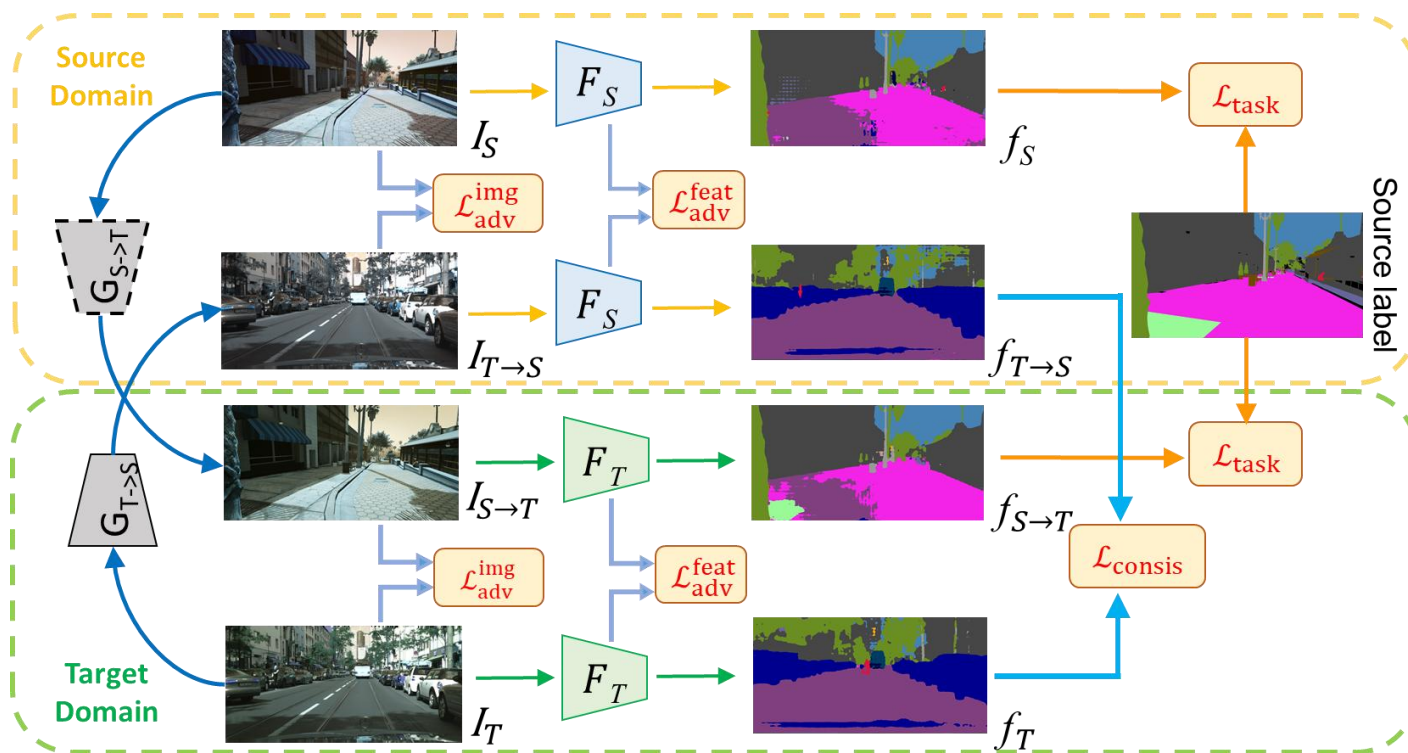
The proposed CrDoCo

- CNN for feature extraction
- Feature-level adversarial learning



The proposed CrDoCo

- Task loss for source images and images translated from source
- Cross-domain consistency loss for target images and images translated from target domain



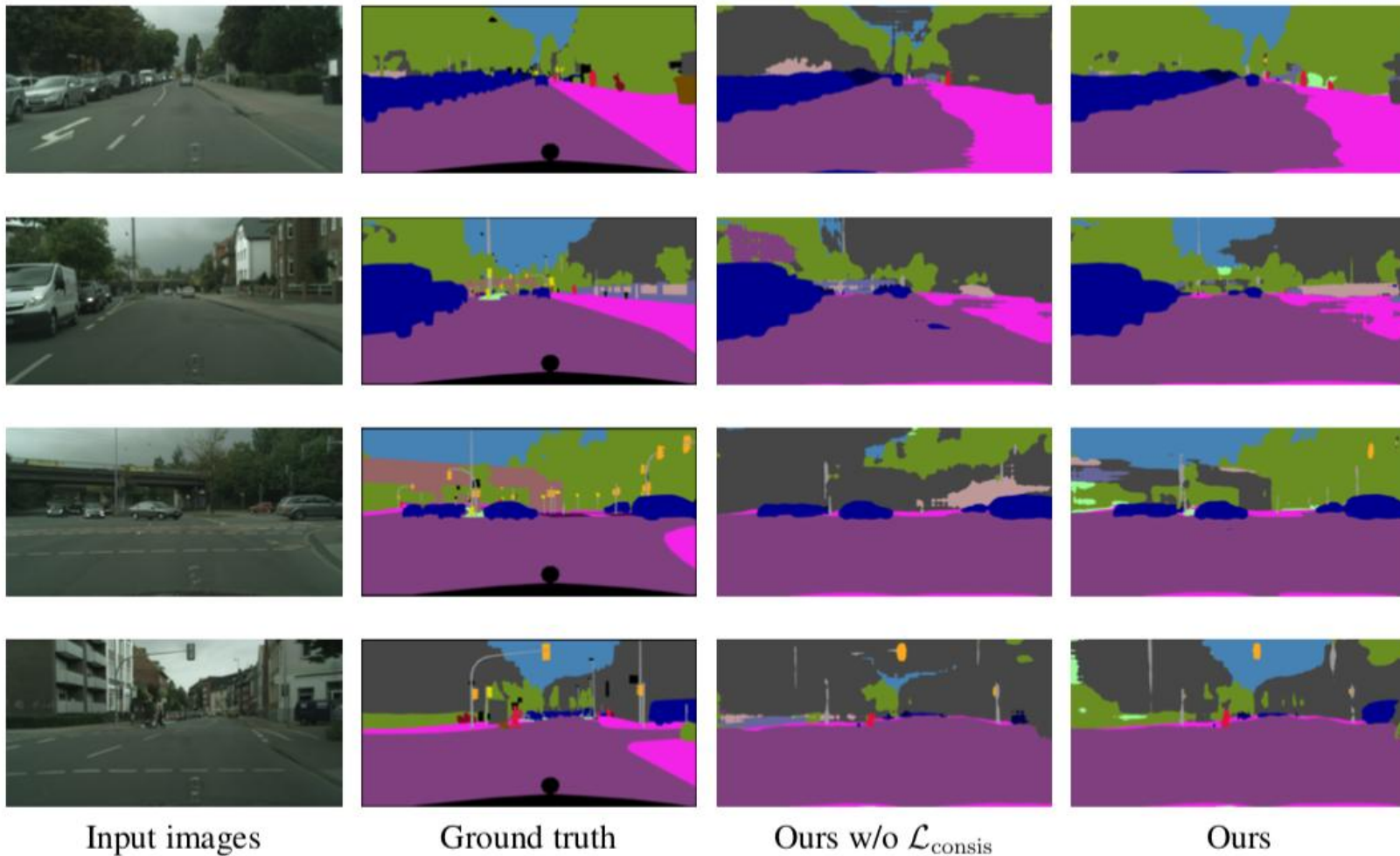
Experiments on semantic segmentation

- GTA5 -> Cityscapes

Method	Backbone	Road	Sidewalk	Building	Wall	Fence	Pole	Traffic Light	Traffic Sign	Vegetation	Terrain	Sky	Person	Rider	Car	Truck	Bus	Train	Motorbike	Bicycle	mean IoU	Pixel acc.
Synth. [7]	DRN-26 [46]	68.9	19.9	52.8	6.5	13.6	9.3	11.7	8.0	75.0	11.0	56.5	36.9	0.1	51.3	8.5	4.7	0.0	0.1	0.0	22.9	71.9
DR [41]		67.5	23.5	65.7	6.7	12.0	11.6	16.1	13.7	70.3	8.3	71.3	39.6	1.6	55.0	15.1	3.0	0.6	0.2	3.3	25.5	73.8
CycleGAN [51]		89.3	<u>45.1</u>	<u>81.6</u>	<u>27.5</u>	18.6	29.0	<u>35.7</u>	17.3	79.3	<u>29.4</u>	71.5	59.7	15.7	<u>85.3</u>	18.2	14.8	1.4	21.9	12.5	39.6	86.6
UNIT [24]		<u>90.5</u>	38.5	81.1	23.5	16.3	30.2	25.2	18.5	79.5	26.8	<u>77.8</u>	59.2	<u>17.4</u>	84.4	<u>22.2</u>	16.1	1.6	16.7	<u>16.9</u>	39.1	87.1
FCNs ITW [15]		70.4	32.4	62.1	14.9	5.4	10.9	14.2	2.7	79.2	21.3	64.6	44.1	4.2	70.4	8.0	7.3	0.0	3.5	0.0	27.1	-
CyCADA [14]		79.1	33.1	77.9	23.4	17.3	<u>32.1</u>	33.3	31.8	81.5	26.7	69.0	<u>62.8</u>	14.7	74.5	20.9	<u>25.6</u>	6.9	18.8	20.4	39.5	82.3
DS [7]		89.0	43.5	81.5	22.1	8.5	27.5	30.7	18.9	<u>84.8</u>	28.3	84.1	55.7	5.4	83.2	20.3	28.3	0.1	8.7	6.2	38.3	87.2
GAM [17]		-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	<u>40.2</u>	81.1
Ours w/o $\mathcal{L}_{consis}$		89.1	44.9	80.9	27.5	18.8	30.2	35.6	17.1	79.5	27.2	71.6	59.7	16.1	84.6	18.1	14.6	1.4	22.1	10.9	39.4	85.8
Ours		95.1	49.2	86.4	35.2	22.1	36.1	40.9	<u>29.1</u>	85.0	33.1	75.8	67.3	26.8	88.9	23.4	19.3	<u>4.3</u>	25.3	13.5	45.1	89.2
Synth. [47]	FCN8s [25]	18.1	6.8	64.1	7.3	8.7	21.0	14.9	16.8	45.9	2.4	64.4	41.6	17.5	55.3	8.4	5.0	6.9	4.3	13.8	22.3	-
Curr. DA [47]		74.9	22.0	71.7	6.0	11.9	8.4	16.3	11.1	75.7	13.3	66.5	38.0	9.3	55.2	18.8	<u>18.9</u>	0.0	16.8	16.6	28.9	-
LSD [35]		88.0	30.5	78.6	25.2	23.5	16.7	23.5	11.6	78.7	27.2	71.9	51.3	19.5	80.4	19.8	18.3	0.9	20.8	18.4	37.1	-
Ours		89.1	33.2	80.1	26.9	25.0	<u>18.3</u>	<u>23.4</u>	<u>12.8</u>	<u>77.0</u>	29.1	72.4	55.1	20.2	<u>79.9</u>	22.3	19.5	<u>1.0</u>	<u>20.1</u>	18.7	38.1	86.3



Experiments on semantic segmentation

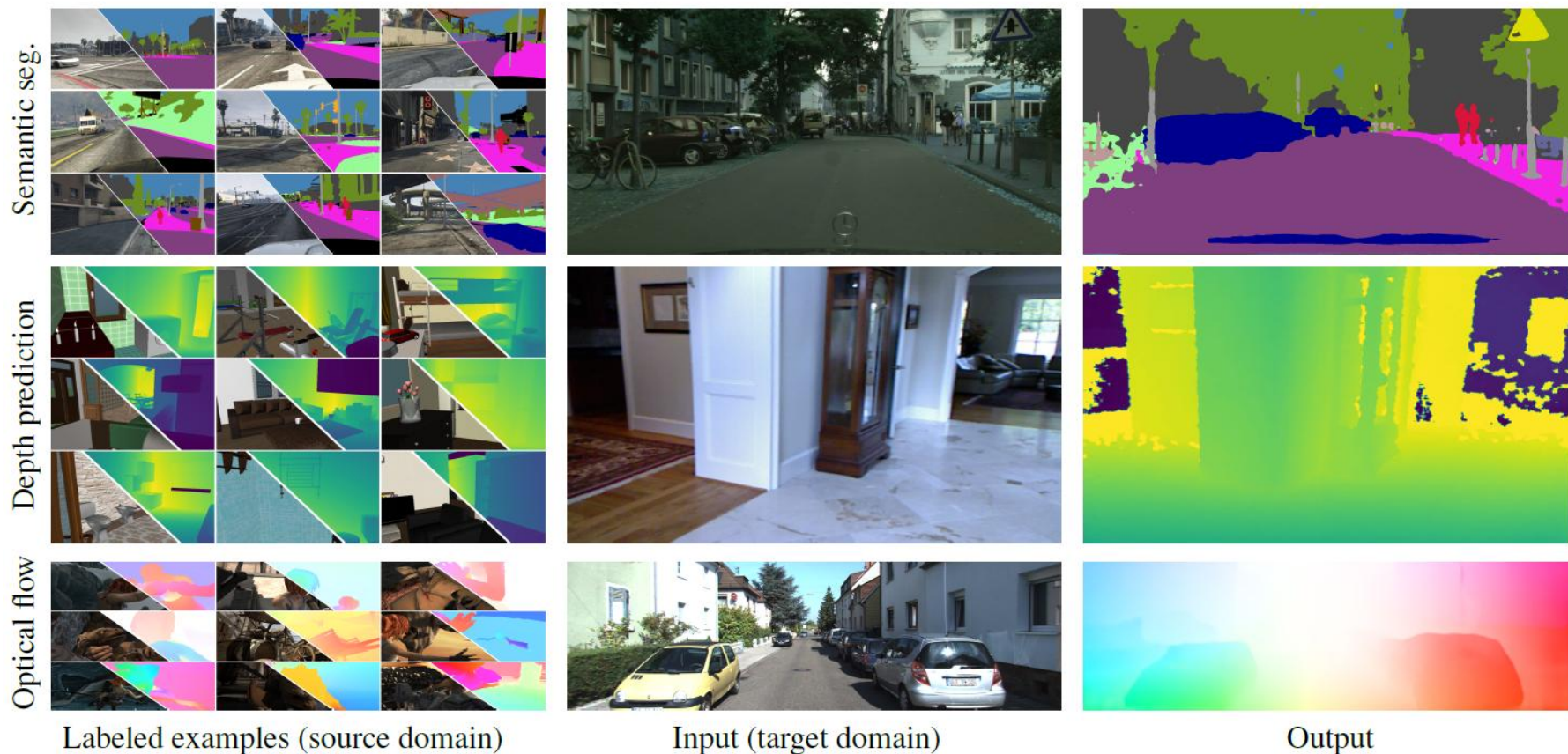


More applications

Code will be available at:

<https://github.com/YunChunChen/CrDoCo-pytorch>

- Depth estimation and optical flow estimation



References

- **Adversarial learning** for semi-supervised semantic segmentation
 - Adversarial Learning for Semi-supervised Semantic Segmentation (BMVC'18)
by W.-C. Hung et al.
- **Multiple instance learning** for weakly supervised semantic segmentation
 - Weakly Supervised Instance Segmentation using the Bounding Box Tightness Prior (NeurIPS'19)
by C.-C. Hsu et al.
- **Transfer learning** for segmentation training data synthesis
 - CrDoCo: Pixel-level Domain Transfer with Cross-Domain Consistency (CVPR'19)
by Y.-C. Chen et al.



Thank You for Your Attention!

THANK YOU FOR YOUR ATTENTION!

Yen-Yu Lin (林彥宇)

Email: lin@cs.nycu.edu.tw

URL: <https://www.cs.nycu.edu.tw/members/detail/lin>

